

DMC-B140-M

Manual Rev. 1.0adev



本手册的内容是针对 Galil 的运动控制器 DMC-B140-M (固件版本 1.0adev) 编写。Galil 的运动控制器虽然都使用相同的指令与编程语法，但是不同的控制器之间还是有一定的差别。如果您使用的是其它型号的 Galil 运动控制器，则本手册中的内容对您而言，可能是不完整的，甚至是错误的。

由于我们能力有限，虽然尽力避免，但本手册中的内容仍可能包含错误。对此我们深感抱歉，也希望您能谅解！

目录

第一章 概述	1
简介	1
DMC-B140-M 功能组件:	1
微处理器	1
通讯接口	1
电机接口	2
通用 I/O	2
第二章 设置	3
硬件设置 (跳线)	3
MRST:	3
UPGD:	3
19.2:	3
MO:	3
ABORT:	4
A-EN、B-EN 与 C-EN、D-EN:	4
建立通讯连接	4
为 DMC-B140-M 连接电源	4
在 PC 上安装通讯软件	4
与 GALIL 软件建立通讯	5
软件设置 (参数)	6
串口回显:	6
主动信息的输出方式:	6
串口主动信息高位设置:	7
串口发送失败的处理:	7
网络地址:	7
工作周期:	7
急停输入:	8
限位输入:	8
限位输入方式:	8
输入编码器信号:	9
保存对状态的修改:	9
恢复为上次保存的状态:	9
恢复为出厂状态:	9
第三章 连接	10
概述	10
电源接口	10
驱动器接口	10
DC5V 电源输出	10
脉冲指令输出	11
编码器信号输入	11
使能输出	11
其它输入/输出信号接口	12
输出信号接口	12
输入信号接口	13
第四章 通讯	14
说明	14
RS-232 接口	14
引脚定义	14

RS-232 配置	14
Ethernet 接口	14
通讯协议	14
地址分配	15
与多个设备通讯	16
第五章 指令与程序	17
说明	17
指令格式	17
执行指令	17
控制器对指令的回应	18
操作数	19
程序基础	19
程序的组成	19
程序的输入与编辑	19
程序的下载与运行	20
控制器已下载程序的查看	20
程序的保存	20
第六章 控制	21
概述	21
I/O	21
I/O 控制指令汇总	22
独立定位	22
独立定位指令汇总	22
举例:	23
JOG 运动	24
JOG 运动指令汇总	24
举例:	25
位置跟随	25
位置跟随指令汇总	26
举例:	26
多维直线插补	28
插补矢量轴	28
指定直线段	29
附加指令	29
直线插补指令汇总	30
举例:	31
平面直线和圆弧插补方式	32
矢量路径	32
附加指令	33
平面插补控制指令汇总	34
举例	34
电子齿轮同步功能	35
电子齿轮同步功能指令及操作数汇总	37
举例	37
电子凸轮	39
电子凸轮指令汇总	41
举例	41
轨迹方式（样条插值）	42
指定轨迹线段	42
轨迹方式指令汇总	43
虚拟轴功能	43

运动平滑(S 曲线加减速).....	44
用 IT 指令使运动平滑:	44
举例	44
用 KS 指令使输出脉冲频率变化平缓	45
原点返回功能	45
举例	47
原点返回指令汇总	47
位置锁存（高速位置捕捉）功能	48
高速位置捕捉指令汇总	48
第七章 编程	50
概述	50
程序格式	50
标号与注释	50
特殊标号	51
注释程序	51
运行程序——多任务	53
调试程序	54
跟踪指令	54
错误处理指令	54
停止代码指令	54
存储空间查询指令	54
操作数	55
举例	55
程序流程指令	55
事件触发	55
跳转 JP	59
条件判断 IF、ELSE、ENDIF	60
子程序	61
堆栈变换	62
自动启动子程序	62
用于监测条件的自动子程序	62
数学及函数表达式	65
数学运算符	65
位运算符	65
函数	66
变量	67
变量名	67
变量赋值	67
以变量作为指令参数	68
输出变量值	68
操作数	68
数组	69
定义数组	69
数组元素赋值	69
用变量指定数组元素地址	69
上载、下载数组到板上存储器	70
将数据自动记录到数组中	70
数据输入（数字和字符）	71
数据输出（数字和字符）	72
输出信息	72
信息函数格式化字符汇总	73
显示变量和数组	73

查询指令	74
对变量和数组元素格式化	75
输入中断功能	76
应用举例.....	76
线缆裁切器	76
X-Y-Z 工作台控制器（3 轴应用）	77
第八章 防护	80
说明	80
硬件防护.....	80
输出保护	80
输入保护	80
软件防护.....	81
位置限制（软件限位）功能.....	81
位置极限信号处理子程序	81
附录.....	83
DMC-B140 控制器数据记录集	83
DMC-B140 控制卡规格	83

第一章 概述

简介

DMC-B140-M 为经济型以太网接口独立式四轴运动控制器。控制器本身提供了四轴电机控制接口以及通用 I/O 信号接口，并提供了用户程序、数据的存储空间。DMC-B140-M 控制器可独立工作，也可利用通讯接口与其它控制器、人机界面配合使用，实现功能更复杂、操作更便利的综合控制。

DMC-B140-M 提供一个以太网接口用于实现大数据量、多设备之间的通讯，也提供了一个 RS232 接口用于简单设备的连接。

DMC-B140-M 控制器输出脉冲/方向信号，可控制步进及伺服电机实现单轴移动、多轴插补、样条插值、电子齿轮、电子凸轮等多种运动模式。同时为用户提供了位置反馈信号接口，可实现对实际运动状态的检测及补偿。控制器为每轴提供独立的前、后极限以及原点信号接口。

DMC-B140-M 的用户指令集包括一百多条指令以及函数。用户可通过这些指令实现参数设置、状态查询、数学和逻辑运算、运动控制、I/O 控制等功能。用户可以将控制程序、数据直接保存在 DMC-B140-M 控制器上。

DMC-B140-M 功能组件：

DMC-B140-M 控制器可划分为以下功能组件：

微处理器

DMC-B140-M 处理器部分的核心是专用的飞思卡尔 32 位微处理器。处理器实现运动规划，对指令的解释，对输入信号的处理等功能。控制器采用与其他 GALIL 控制器相同的指令系统，为用户提供了包括指令、函数、系统变量在内的两百多个保留字。同时还为用户提供了存储空间，包括用户程序存储区（450 行×40 字）、独立变量存储区（126 个变量）、数组存储区（一维数组 6 个，共计 800 个数组元素）。

通讯接口

通讯接口用户接收上位控制器的指令，与其它控制部件进行数据交换。DMC-B140-M 的通讯接口包括一个 10/100M 以太网接口用于实现大数据量、多设备之间的通讯。而通过外加一个无线 AP，就可以实现无线通讯。控制器也提供了一个 RS232 接口用于简单设备的连接，波特率为 19200/115200。

电机接口

DMC-B140-M 控制器为每轴电机提供了脉冲/方向信号输出、驱动器使能信号输出、增量编码器信号输入以及前后极限信号、原点开关信号输入。

控制器输出脉冲/方向信号，可用于控制步进电机与伺服电机。脉冲信号频率最高为 3MHz，可以满足绝大多数高速、高分辨率的应用需求。

控制器输出驱动器使能信号，可用于步进驱动器的半流或脱机控制，也可用于伺服驱动器实现电机的打开与关闭。

控制器每轴可以接收一组增量编码器信号。该信号可以是正交方波的增量编码器信号，也可以是脉冲/方向信号。对于正交方波信号，可接收的最高频率为 12MHz（四倍频后）；对于单脉冲信号，可接收最高频率为 3MHz。控制器可接收编码器的零位信号（零位信号宽度不得小于 80ns）。

控制器每轴提供独立的前、后极限信号输入以实现行程保护，并提供原点开关输入以确定机械原点位置。

通用 I/O

DMC-B140-M 为用户提供 8 路通用数字输入，用户可以按照自己的应用需求自行编程定义它们的功能。这些输入点也可以被定义为各轴的高速位置捕捉及独立急停信号输入。

DMC-B140-M 还有 4 路通用数字输出。用户可以用这些输出实现自己所需要的功能。

第二章 设置

本章介绍用户可以根据自己的控制要求对 DMC-B140-M 进行哪些基本设置，以及如何实现这些设置。

硬件设置（跳线）

MRST:

（待插入照片）

在 DMC-B140-M 控制器上，JP1 中的第一个（左边）是标为 MRST 的跳线。MRST 跳线用于主复位（恢复到出厂时状态）。当 MRST 短接时，对控制器上电或按下复位输入按钮时，控制器就执行主复位操作。无论何时对控制器进行主复位，都将清除用户存储在 EEPROM 中的所有程序、数组、变量，并将控制器参数恢复到出厂设置。

正常使用中，此跳线应处于开放状态。

UPGD:

在 DMC-B140-M 控制器上，JP1 中的第 2 个为 UPGD 跳线。UPGD 短接时可以对控制器强制执行固件下载。

在正常状态下，对控制器下载固件不需要短接此跳线。如果在下载固件过程被中断，会使 EEPROM 内数据不完整，以至于控制器不能正常工作，甚至无法正常与 PC 通讯。在这种情况下，就需要短接此跳线，对控制器强制执行固件下载，以使控制器恢复正常工作。

正常使用中，此跳线应处于开放状态

19.2:

DMC-B140-M 默认串口波特率为 115200。可以通过 JP1 中的第 3 个标注为 19.2 的跳线改变串口通讯波特率。当此跳线短接时，串口波特率为 19200。

此跳线出厂时为开放状态，用户使用中根据所需要的串口波特率来设置此跳线状态。

MO:

在 DMC-B140-M 控制器上，JP1 中的第四个为 MO 跳线。可以通过此跳线选择控制器开机（复位）后的电机状态。短接 MO 位置上的跳线，就会使控制器在上电时保持“电机关闭”状态。为了使打开电机，就需要执行 SH 指令。不短接该跳线，电源一上电，控制器使电机处于之前保存的状态（默认为电机打开状态），要关闭电机，就需要执行 MO 指令。

无论电机/驱动器的打开和关闭是否受 DMC-B140-M 控制，在控制器上当某轴处于关闭状态时，控制器不会执行针对该轴的运动指令。

此跳线出厂时为开放状态，用户使用中根据控制需要来设置此跳线状态。

ABORT:

(待插入照片)

此跳线在 DMC-B140-M 左上部分。用于设置急停输入（通用输入 8）的响应状态。默认为常开输入状态（N.O.）。在用户将此输入点设置为急停输入时，如外部输入信号截止或开路时，控制器正常执行运动指令；外部输入信号导通时，控制器各轴停止运动，且不执行任何运动指令。

用户如果要提高系统的安全性，可将其设置为常闭输入状态（N.C.），即只有在外部信号输入导通的情况下，才能正常执行运动指令。

此跳线出厂时为 N.O. 状态，用户可根据需要设置此跳线状态。

A-EN、B-EN 与 C-EN、D-EN:

这四个跳线用于设置 DMC-B140-M 控制器各轴在使电机关闭或使能状态时的电气状态。默认为 SH-CLOSE。此状态下，电机打开时，接口中的 EN+与 EN-导通；电机关闭状态下，接口中的 EN+与 EN-截止。如用户设置为 MO-CLOSE，则电机打开时，接口中的 EN+与 EN-截止；电机关闭时，接口中的 EN+与 EN-导通。

作为一种电路防护措施，EN+与 EN-反向永远处于导通状态。

此跳线出厂时为 SH-CLOSE 状态，用户可根据需要设置此跳线状态。

建立通讯连接

除硬件设置外，控制器的一些功能需要用软件进行设置，下面首先介绍软件设置前的准备工作。

为 DMC-B140-M 连接电源

DMC-B140-M 需要使用 DC24V（DC19—33V）作为工作电源。DMC-B140-M 的电源接头有三个引脚，V+连接外部 DC24V 电源的正端，V-连接外部 DC24V 电源的负端（或称 COM 端），第三脚用于保护接地。

如电源电压低于 DC19V，或 V+/V-接反，则控制器处于保护状态，无法工作。如接入电源电压高于 33V，可能导致控制器损坏。

对于 DC24V 电源，在无外部负载的情况下，控制器消耗电源不大于 0.3A。如电源电压低于 24V，则电流相应增加。

在 PC 上安装通讯软件

原则上，任何可以实现串口或网口自由通讯的软件，例如 WINDOWS 系统提供的 超级终端 程序，都可以用于 PC 与 DMC-B140-M 控制器的通讯。在这里，我们推荐使用 GALIL 公司提供的

GalilTools 软件。使用此软件可以是用户暂时不必关心一些通讯设置方面的细节。同时，这个软件针对 GALIL 控制器的应用提供了一些非常有用的辅助功能。GalilTools 软件分别提供运行于 WINDOWS 下的版本以及运行于 Linux 下的版本。（本手册均以 WINDOWS 系统为例进行说明）

在安装 GalilTools 软件的过程中，会询问是否安装 Galil 控制卡的 PCI 接口驱动程序。由于 DMC-B140-M 本身不使用 PCI 接口，所以是否安装都没有影响。如果用户还要使用 GalilTools 与 DMC-18xx 系列的运动控制器通讯，则必须安装 PCI 驱动。

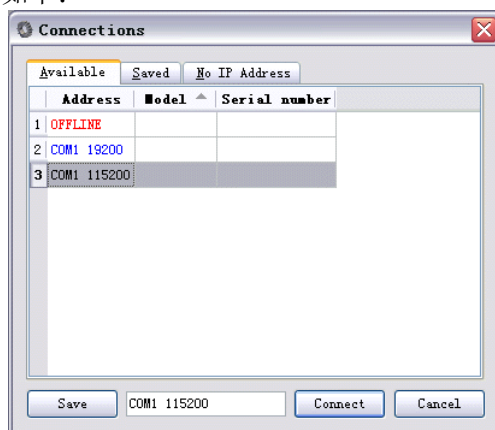
如果用户已经习惯使用早期版本的 GALIL 软件，如 SmartTerminal，也可以用自己惯用的软件实现基本的参数设置。不过，由于早期的软件不能识别 DMC-B140-M 控制器，所以在软件中将控制器信号设置为 DMC-2xxx。软件只能使用串口与控制器通讯，而且一些辅助功能，如显示 Datarecoder，无法实现。

与 GALIL 软件建立通讯

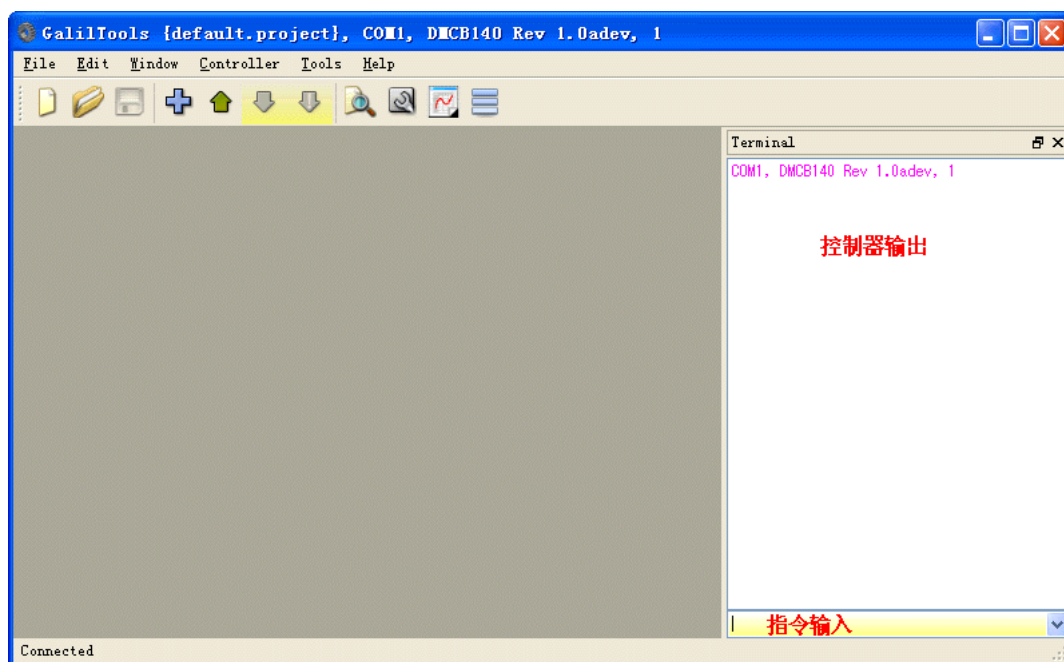
关于 DMC-B140-M 的通讯，将在第四章详细介绍。本章只是采用最简单的步骤使用户可以对控制器进行基础的参数设置。下面的步骤是以 GalilTools 软件的串口连接进行说明。

如用户使用的不是 PC 的标准串口，而是由其它方式转换得到的 RS232 接口，要首先确认该串口中包含硬件握手信号，并支持所选定的波特率。

连接 PC 与控制器的串口通讯线，应使用两边为 DB-9 母接头的直通 9 芯线。将控制器与 PC 的 RS232 接口连接。开机后，在 PC 上执行 GalilTools 软件。软件启动后，会出现连接窗口，窗口中会列出所有已经连接的 Galil 控制器以及可用于连接的串口和通讯速率。最简单的情况下，PC 暂时未连接任何 Galil 控制器，则窗口内容如下：



选中所用的串口与传输速率后，点击“Connect”建，就可以在 PC 与控制器之间建立软件通讯连接。相应连接建立后，会在窗口标题以及控制器输出区中显示连接方式（COM1）、控制器型号（DMC-B140）、内部固件版本（1.0adev）以及控制器的序列号（1）。



软件设置（参数）

建立通讯后，就可以在 GalilTools 软件界面右下输入 Galil 指令，对控制器的参数进行设置。有关 Galil 指令的更多内容，请阅读第五章以及指令手册，在此并不详细介绍。用户需要修改哪些状态参数的设置时，只要严格按照下面给出的内容输入就可以了。输入过程中要注意：每个指令均以 <enter>键结束；所有字母均为大写；一行指令中的任何位置不要加入空格；注意区分字母 O 与数字 0，字母 I 与数字 1；不要开启中文输入法。

以下是一些控制器的基本状态，用户可以根据需要修改其中的部分或全部内容。

串口回显：

控制器在收到任何从串口发来的信息后，都会首先将该信息原样发送回去。这样给信息的发送者进行校对的机会，可以减少由于信息传输错误带来问题的可能性。

如用户不需要这一功能，则输入指令：

EO0

如用户需要控制器恢复这一功能，则输入指令：

EO1

主动信息的输出方式：

控制器从通讯口接收到的信息时，从哪个通道接收的信息，也会在哪个通道输出相应的回应信息。但是控制器本身也会主动产生一些输出信息，例如出错信息，以及程序中设置输出信息。这些信息称为主动信息或非请求信息。在出厂状态下，控制器优先将这些信息通过串口发送。

如用户需要控制器优先使用网口发送主动信息，则输入指令：

CFA

如用户需要恢复控制器优先使用串口发送主动信息，则输入指令：

串口主动信息高位设置:

Galil 控制器串口通讯中, 每个字节包含 8 个数据位。ASCII 字符只须要 7 个数据位来描述, 最高位通常为 0。控制器可以将主动信息每个字节的最高位设置为 1, 这样, 与控制器通讯的设备就可以通过最高位来区分收到的信息是对指令的回应, 还是控制器的主动信息。

如用户需要将主动信息的字节最高位设置为 1, 则输入指令:

CW1

如用户不需要将主动信息的字节最高位设置为 1, 则输入指令:

CW2

串口发送失败的处理:

Galil 控制器串口通讯中, 如发送信息时得不到正确的握手信号, 则信息不能发出。用户可以指定在这种情况下, 控制器如何处理: 中断内部程序的执行, 直到串口通讯正常, 信息被发出; 继续执行内部程序, 不再发送该信息。

如用户需要等待直到信息被发出, 则输入指令:

CW, 0

如用户取消信息发送, 继续执行程序, 则输入指令:

CW, 1

网络地址:

控制器出厂默认使用动态网络地址 (DHCP), 如控制器所连接的网络中有支持 DHCP 的服务器, 则每次接如网络时, 服务器会为控制器自动分配临时的 IP 地址, 无须用户干预。如控制器所连接的网络本身不支持 DHCP, 则必须为控制器指定一个固定 IP 地址。

如用户希望控制器使用固定地址, 则输出指令:

DH0

之后, 输入指定的 IP 地址。例如用户希望指定的 IP 地址为 192.168.1.55, 则输入:

IA192, 168, 1, 55

如用户希望控制器恢复使用动态地址, 则输入指令:

DH1

工作周期:

DMC-B140-M 控制器出厂状态下, 内部工作周期约为 1ms (967us)。控制器的内部的系统工作包括对运动轨迹的规划, 对大部分外部信号的响应等等。这些工作每个周期进行一次。用户可以按照自己的要求对这个周期进行一些调整。周期越短, 则控制器的响应速度越快。不过, 在同样的时间内, 工作周期越短, 则用于执行内部系统工作的次数约多, 用于执行用户指令、程序的时间就约少。

如用户希望控制器以最小周期工作, 约为 250us, 则输入:

TM250

如用户需要恢复出厂默认的工作周期，则输入：

TM1000

其它设置可以参考指令手册中的 TM 指令。

急停输入：

控制器的通用输入点中的第 8 路，出厂时默认作为急停输入信号。控制器在急停信号有效时，会立即停止当前的运动，并不再执行新的运动指令。

如用户不需要急停输入信号，只将该路输入作为通用输入使用，则输入：

AX0

如用户需要恢复急停输入信号，则输入：

AX1

限位输入：

控制器为每轴设置了独立的前、后极限输入信号接口。如用户的设备上没有相应的硬件，可以设置某轴的某个极限信号输入无效，将该输入信号做为通用输入使用（在此状态下，用户依然可以设置软极限）。由于四轴的状态需要分别设置，所以这个指令有 4 个参数，顺序对应 A、B、C、D 四轴的状态，参数之间用逗号“,” 分开（第一个参数之前以及最后一个参数之后不需要逗号）参数值代表的意义如下：

- 0 前后极限均有效（出厂默认状态）
- 1 前极限无效，后极限有效
- 2 前极限有效，后极限无效
- 3 前后极限均无效

如用户需要设置 A、B 轴极限信号有效，C 轴前极限无效，D 轴极限无效，需要输入：

LD0, 0, 1, 3

如用户需要恢复到所有极限信号均有效的默认状态,需要输入

LD0, 0, 0, 0

限位输入方式：

控制器为每轴设置了独立的前、后极限输入信号接口。当极限信号有效时，如电机正在向该方向运动，则控制器会使电机减速停止。电机停止后，则不允许再次向该方向运动，直到该极限信号无效。

出厂状态下，控制器是以对应输入接口导通的状态为极限有效状态（常开输入）。如用户要提高系统的安全性，可以设置为对应输入接口截止的状态为极限有效状态（常闭输入）。

如用户需要设置极限信号为截止/开路状态下有效，输入指令：

CN1

如用户需要恢复极限信号为导通状态下有效，输入指令：

CN-1

输入编码器信号：

DMC-B140-M 控制器每轴可接受一组编码器反馈信号，A+、A-、B+、B-、Z+、Z-，默认状态下，A、B 为正交方波信号，Z 为零位信号（ZERO，也称标志或索引信号，INDEX）。

控制器也可以将 A 理解为脉冲，B 理解为方向的形式对反馈信号计数。设置指令有四个参数，顺序对应 A、B、C、D 轴的状态设置，参数值代表的意义如下：

- 0 正交方波信号，以 A 相位超前为正
- 1 脉冲/方向信号，以方向信号高电位为正
- 2 正交方波信号，以 B 相位超前为正
- 3 脉冲/方向信号，以方向信号低电位为正

如用户需要设置 A、B 轴为默认状态，C 轴为正交方波信号，但方向取反，D 轴为脉冲/方向信号，需要输入：

CE0, 0, 2, 1

如用户需要恢复到控制器出厂默认状态，需要输入

CE0, 0, 0, 0

保存对状态的修改：

用户按照需要调整控制器的以上参数后，可以将当前的设置状态保存。这样，下次开机后就不需要重新设置一次（需确认 MRST 跳线未短接）。同时被自动保存的还有其它参数。控制器的全部指令与参数，请参看 DMC-B140-M 指令手册。

用户要保存设置，输入指令：

BN

恢复为上次保存的状态：

如用户只是临时调整参数，调整后还要恢复到原有的参数，可以对控制器执行复位操作。单纯的复位会将所有参数恢复到上次保存的状态。实现控制器复位可以通过关闭控制器电源再重新接通、按下控制器左上方侧面的 RESET 按钮实现。也可以发送指令：

RS

（执行复位操作时，要确认 MRST 跳线未短接。）

恢复为出厂状态：

当控制器执行复位动作时，如 MRST 跳线处于短接状态，则控制器执行主复位动作，所有参数恢复到出厂状态。

第三章 连接

概述

DMC-B140-M 控制器为电机控制提供了脉冲信号输出、编码器信号输入、驱动器使能输出，而且为每轴提供了前后极限以及原点信号输入。控制器还提供了 8 个通用输入和 4 个通用输出以及一个状态输出信号用于 I/O 控制。

本章为您介绍控制器上的输入和输出信号的电气规格以及如何在控制器与外部设备之间连接这些信号。

电源接口

DMC-B140-M 需要使用 DC24V（DC19—33V）作为工作电源。此电源与控制器内部工作电源隔离。

在无外部负载的情况下，控制器消耗电源不大于 0.3A。如用户需要控制器为编码器等外部器件提供 5V 电源，则建议采用 0.5A 以上规格 DC24V 电源。如电源电压低于 24V，则消耗电流相应增加。

DMC-B140-M 的电源接头有三个引脚，V+连接外部 DC24V 电源的正端，V-连接外部 DC24V 电源的负端（或称 COM 端），第三脚用于防护接地。如电源电压过低或接反，则控制器处于保护状态，无法工作。如接入电源电压过高，可能导致控制器损坏。

驱动器接口

DMC-B140-M 的四个驱动器接口，为四个 DB15 的 D 型接头，依次标注为 A、B、C、D。每个接口内各引脚定义如下：

1	NC	未连接	9	EN-	使能信号负端
2	EN+	使能信号正端	10	GND	DC5V 电源
3	5V	DC5V 电源	11	A+	编码器反馈信号
4	A-	编码器反馈信号	12	B+	编码器反馈信号
5	B-	编码器反馈信号	13	Z+	编码器反馈信号
6	Z-	编码器反馈信号	14	STEP-	脉冲信号的反向输出端
7	DIR-	方向信号的反向输出端	15	STEP+	脉冲信号的正向输出端
8	DIR+	方向信号的正向输出端			

DC5V 电源输出

控制器在每个驱动器接口中提供一组 DC5V 电源输出接点，可为用户连接外部的编码器提供电

源。此电源电压为 5.1V，控制器可提供的全部电流为 0.5A。

脉冲指令输出

STEP+、STEP-、DIR+、DIR-为控制器提供的脉冲/方向信号，用于控制步进电机或工作在位置模式下的伺服电机时。信号输出为长线驱动形式（RS422/TTL）。

绝大部分电机驱动器均可接收线驱动脉冲信号。对于这样的驱动器，只需用屏蔽双绞线将控制器与电机驱动器上的 STEP+、STEP-、DIR+、DIR-对应连接即可。连接时注意：STEP+、STEP-使用一对双绞线；DIR+、DIR-使用一对双绞线；线缆的屏蔽层与控制器接头的外壳连接。

如电机驱动器不能接收线驱动信号，只能接收 5V 的脉冲与方向信号，同时提供正公共端或负公共端。需要将 STEP+、DIR+连接到驱动器的信号接收端，STEP-、DIR-悬空，将 5V 连接到驱动器的正公共端或将 GND 连接到驱动器的负公共端。

如电机驱动器只能接收 12~24V 的脉冲信号，那么 DMC-B140-M 控制器的信号不能直接与该驱动器连接。请更换电机驱动器规格或使用信号转换器件将控制器输出信号转换为驱动器可接收的形式。

编码器信号输入

控制器为每轴提供一组编码器信号输入，可以接收旋转编码器、光栅尺或其它增量位置编码器的 TTL/RS422 信号。

编码器输入口含有 A+、A-、B+、B-、Z+、Z-接点。为避免由于电磁干扰带来的尖峰电压对控制器接收信号的损坏，在 DMC-B140-M 中对所有的编码器输入信号进行了限压保护，保证接受到的信号电压在 0—5V 以内。如外部的编码器信号电压高于 5V，则不能直接与控制器相连接，请更换编码器规格或使用转换部件将信号电压转换为 5V 规格。

控制器默认接收正交方波信号，也可以接收脉冲/方向信号，设置的方式参见第二章的内容，或《指令手册》中的 CE 指令。

编码器信号可接收的最高脉冲频率为 3MHz（对于正交方波，控制器内部执行四倍频处理，最大信号频率为 12 MHz），超过此频率的信号，控制器不能正确处理。而零位信号也要确保宽度不小于 80ns。

由于差分（长线驱动）信号配合双绞线后，在信号的传输距离与传输频率上有明显的优势，所以推荐使用差分输出的反馈信号。对于这样的信号，只需要对应连接 A+、A-、B+、B-、Z+、Z-既可。

如果用户的编码器只有 A、B、Z 信号输出，在信号传输距离较短的情况下也可以正常工作。此时将信号 A 连接到控制器的 A+，信号 B 连接到控制器的 B+、信号 Z 连接到控制器的 Z+，并将编码器的电源负端与控制器的 GND 连接即可。控制器的 A-、B-及 Z-输入保持开路。

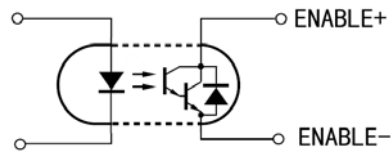
对于 DMC-B140-M 控制器，编码器输入信号为可选连接，用户可以根据自己的应用确定是否连接编码器信号。

使能输出

DMC-B140-M 的使能输出信号为光耦的导通/截止输出。在默认状态下，当控制器使电机打开时，

EN+与 EN-导通，电流可以由 EN+流向 EN-；当控制器使电机关闭时，电流不能从 EN+流向 EN-。如驱动器的控制方式与 DMC-B140-M 的控制方式不一致，可参阅第二章的内容通过跳线对使能信号的输出进行设置。

（注意，作为一种电路保护措施，在任何情况下，电流可以由 EN-流向 EN+。）



指令 SH、MO 可用来直接控制使能信号的输出，具体内容，请参阅《指令手册》中对这两个指令的说明。

对于步进电机驱动器或工作在位置模式下的伺服电机驱动器而言，是否需要连接使能信号，可以依据实际应用需求来确定。无论用户是否连接了驱动器使能信号，当控制器上某轴电机处于关闭状态时，控制器不能执行针对该轴的运动指令。

其它输入/输出信号接口

在 DMC-B140-B 控制器中，将其它输入/输出信号集中在一个 DB37 的接口中。其中各引脚定义如下：

1	RDY-	就绪状态输出负端	20	RDY+	就绪状态输出正端
2	OUT4-	通用输出 4 负端	21	OUT4+	通用输出 4 正端
3	OUT3-	通用输出 3 负端	22	OUT3+	通用输出 3 正端
4	OUT2-	通用输出 2 负端	23	OUT2+	通用输出 2 正端
5	OUT1-	通用输出 1 负端	24	OUT1+	通用输出 1 正端
6	V-	DC24V 电源	25	V+	DC24V 电源
7	V-	DC24V 电源	26	V+	DC24V 电源
8	INCOM	输入信号公共端	27	D_RL	D 轴反向极限信号输入
9	C_RL	C 轴反向极限信号输入	28	D_HM	D 轴原点信号输入
10	C_HM	C 轴原点信号输入	29	D_FL	D 轴正向极限信号输入
11	C_FL	C 轴正向极限信号输入	30	B_RL	B 轴反向极限信号输入
12	A_RL	A 轴反向极限信号输入	31	B_HM	B 轴原点信号输入
13	A_HM	A 轴原点信号输入	32	B_FL	B 轴正向极限信号输入
14	A_FL	A 轴正向极限信号输入	33	AB/IN8	急停输入（通用输入 8）
15	IN7	通用输入 7	34	IN6	通用输入 6
16	IN5	通用输入 5	35	IN4	通用输入 4
17	IN3	通用输入 3	36	IN2	通用输入 2
18	IN1	通用输入 1	37	GND	DC5V 电源
19	5V	DC5V 电源			

输出信号接口

所有输出信号采采用达林顿光耦隔离，可直接用于驱动 DC24V 继电器。输出信号有效时，电流可以从正端流向负端。

通用输出

控制器提供 4 路通用输出信号。这些点定义为 OUT1~OUT4。用指令 SB（设置位）、CB（清除位）、OB（输出位）、OP（输出端口）控制这些输出点。这些指令的具体内容，请参考《指令手册》

内容。

就绪状态输出

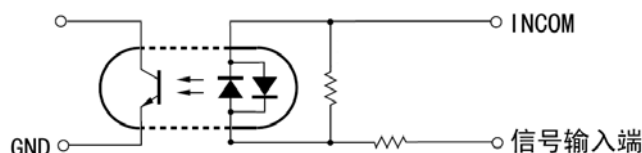
RDY+与 RDY-为控制状态输出。当控制器处于正常工作状态时，此输出信号有效。当控制器未上电，或者上电出现报警时，红色 LED 灯点亮，报警输出 ERROR 变低。以下几种原因可能会导致控制报警：

- 控制器处于开机或复位后的自检过程；
- 控制器内 EEPROM 数据出现错误（保存数据、更新固件的过程中断电可能会导致此问题）；
- 控制器硬件故障

控制器上电后，如此输出无效，控制器上的红色 ERROR 指示 LED 会发光。

输入信号接口

所有输入信号均采用光耦隔离输入。INCOM 为所有输入信号的公共端，当信号输入端与 INCOM 端之间有电压大于 DC18V 时，控制器可以收到对应信号，当电压小于 7V 时，控制器不会收到信号。由于使用的光耦器件为双向输入，所以，无论是源型还是漏型信号源都可以连接。



极限信号

前极限信号有效，就会立即禁止向正方向运动；而后极限信号有效，就会立即禁止向反方向运动。如果在运动过程中，运动方向的极限信号生效，控制器就会以用户所设置的减速度使运动减速而停止。用户可以设置极限信号在何种状态下有效，具体设置方式参见第二章或《指令手册》中 CN 指令的介绍。

在极限信号有效期间，不可能使电机向极限信号所保护的方向运动。

控制器提供操作数使用户可以查询极限信号的当前状态，具体内容，请参阅《指令手册》上对于操作数 LF 以及 LR 的说明。

原点信号

原点信号用于在运动控制中提供机械原点。原点信号状态的变化提示运动物体到达了一个特殊参考点位置。参考点可以是空间中的一点或编码器标志脉冲（Index 或 Zero）。

控制器提供操作数使用户可以查询极限信号的当前状态，具体内容，请参阅《指令手册》上对于指令 HM 的说明。

急停输入

急停输入的功能是，一旦此输入点逻辑状态发生变化就立即使控制器停止电机的运行。在出厂默认状态下，DMC-B140-M 控制器将通用输入的第 8 点作为急停信号。

急停输入的响应方式不同于限位开关输入有效时的情况。当急停输入生效时，控制器立即停止执行运动命令，而限位开关输入生效时，控制器会使电机减速逐渐停止。

通用输入

DMC-B140-M 有 8 个通用输入点，这些输入点可以用做任意信号的输入。用户可以用这此输入点建立与外部事件的关联（第 8 点默认作为急停信号，如用户需要将其作为通用输入使用，必须更改控制器的默认设置，具体方式请参阅第二章）。同时，这些输入点也可以用作特定轴的专用输入。1-4 点可以作为 A-D 各轴的位置锁存输入，其中第 1 点对应 A 轴、第 2 点对应 B 轴……（关于位置锁存功能的详细介绍，参见“控制”一章中相应的部分，以及《指令手册》中 AL、RL 指令的介绍）；第 5-8 点可以作为 A-D 各轴独立的急停输入，其中第 5 点对应 A 轴、第 6 点对应 B 轴……（详见《指令手册》中 CN 指令）。

这些输入点的状态可以用 TI 指令查询，也可以函数@IN[x]分别读取这些输入点状态，其中 x 代表输入点号（1~8）。详见《指令手册》

第四章 通讯

说明

DMC-B140-M 控制器提供一个 DB9 的 RS-232 通讯接口和一个 RJ45 的 10/100M 的 Ethernet 接口。

RS-232 接口

引脚定义

1	NC	6	NC
2	TXD—输出	7	RTS—输入
3	RXD—输入	8	CTS—输出
4	NC	9	NC
5	GND		

RS-232 配置

GALIL 控制器的 RS-232 采用如下接口协议：硬件握手；8 个数据位，1 个起始位，1 个停止位，全双工，无校验。任何设备如需通过串口与控制器通讯，必须按照这个要求配置通讯格式。如果该设备本身不支持硬握手，也可以在 DMC-B140 一侧将接口内的 7、8 脚短接。控制器默认的通信口波特率为 115.2K，可以用控制卡上标注为 19.2 的跳线设置 RS-232 通信口的波特率。

DMC-B140 控制器出厂默认状态下，会对串口收到的 ASCII 文本字符直接发送回去，其它设备可以通过接收此信息检查在传输中是否存在错误，如用户不需要这个功能，可以将此功能关闭（参见第二章）。

通常情况下，DMC-B140 控制器是将串口收到信息存储在通讯缓冲区中，直到收到一个回车符号 (0DH) 后才对缓冲区中的内容进行解析。如控制器对解析出的内容不可理解，则反馈一个问号“?” (3FH)。如果收到的内容为指令，则执行该指令（如指令本身要求反馈信息，则将该信息自串口反馈），而后反馈一个冒号“:” (3AH)。

Ethernet 接口

通讯协议

Ethernet 是以数据包为单位传送信息的本地网络。因此，需要通信协议来规定如何发送、接收这

些包。Galil 控制器支持两种工业标准协议，TCP/IP 和 UDP/IP。控制器以所接收的协议格式进行相应的回应。

TCP/IP 是一种“应答”协议。开始通信前，必须将主站（客户端）与从站（服务器）相连。当接受到的数据包时，会告知发送方“已收到”，若发送方未收到回应，就认为此数据包传送失败，并自动重新发送。

与 TCP/IP 协议不同的是，UDP/IP 本身不需要“应答”。此协议类似于用 RS-232 的方式进行通信。由于此协议本身不提供关于丢失数据的保护，因此，用户必须自己在发送方编程处理这种情况。

尽管 UDP/IP 协议的数据传输效率高效、也更简单，但 GALIL 建议用户使用 TCP/IP 协议。因为 TCP/IP 协议可以保证通讯更为可靠。

Ethernet 通信以数据包传送信息，数据包必须限定在 470 数据字节以内。更大的包可能引起控制器通信阻塞。为了不丢失传送信息，GALIL 建议用户在发送下一个包之前等待包接收的回答信息。（如用户使用 GALIL 提供的通讯软件，则可不必关注这些细节）。

IK 指令阻止控制器从 1000 以下的以太网端口接收数据包，0，23，25，68，80 和 502 端口除外。如需从所有端口接收数据包，需要将 IK 设置为 0。

通常情况下，控制器是将收到信息存储在通讯缓冲区中，直到收到一个回车符号（0DH）后才对缓冲区中的内容进行解析。如控制器对解析出的内容不可理解，则反馈一个问号“?”（3FH）。如果收到的内容为指令，则执行该指令（如指令本身要求反馈信息，则将该信息输出），而后输出一个冒号“:”（3AH）。

地址分配

Ethernet 设备的地址分配包括三层。第一层是 Ethernet 或硬件 MAC 地址。这是一个唯一且永久的 6 字节数。任何设备的 MAC 地址都是由生产厂家设定，而且世界上不会有相同的 MAC 地址。Galil 产品的 MAC 地址末字节与产品序号相同。

通过在 GalilTools 中输入 TH 指令可以查询 DMC-B140 的 Ethernet 地址。

MAC 地址：00-50-4C-28-00-03

第二层是 IP 地址。它是一个 32 位（4 字节）数，例如：192.168.1.15。IP 地址由各局域网络限定分配。DMC-B140-M 控制器的 IP 地址可以由多种方法分配。

DHCP:

DMC-B140-M 控制器默认使用动态 IP 地址，如果局域网中有 DHCP 服务器，则可直接使用此功能。DHn 指令用于设置控制器是否支持 DHCP，参数 n 为 0 或 1。如局域网中没有支持 DHCP 的服务器，则必须使用 DH0 指令来关闭 DMC-B140-M 对 DHCP 的支持（参见上一章的内容）。

BOOT-P:

BOOT-P 只能在 DH 设置为 0 时使用。可以使用内部网络上的 BOOT-P 服务器或 Galil 软件。当打开 Galil 软件时，Galil 软件就搜索连接到当前局域网上所有未分配 IP 地址的 Galil 控制器。用户选定控制器，Galil 软件就会给控制器分配所指定的 IP 地址。此 IP 地址会被保存到（BN）控制器的非易失性存储器中。注意：如果网络中有多个控制器，需要通过序列号区分。

注意：确保局域网中只有一个 BOOT-P 服务器在运行。如果您的网络有 BOOT-P 或 DHCP 服务器在运行，它就可以自动分配 IP 地址给连接到网络中的控制器。为了确保 IP 地址正确无误，在把控制器连接到 Ethernet 网络之前，请与您的系统管理员联系，确认服务器工作方式。

IA 指令:

DMC-B140-M 控制器本身提供了 IA 指令，允许用户直接指定控制器的地址。IA 指令只有在 DH 设置为 0 时有效。输入要分配的 IP 地址。地址为 4 字节数，中间用逗号分开或带符号 32 位数（例如，IA192, 168, 1, 15）。设置好 IP 地址后，输入 BN 并回车，将 IP 地址保存到控制器的非易失性存储器中。（参见第二章的内容）

注意：所选择的 IP 地址应是不需要通过 GATEWAY 存取的地址（GATEWAY 是管理内部网络和外部世界通信的应用软件）。

Ethernet 地址分配的第三层是 UDP 或 TCP 端口号。Galil 控制器不需要指定端口号。每次连接控制器时，端口号由终端或主站建立。应用中的典型端口号如下：

Port 23: Telnet

Port 502: Modbus

Port 80: HTTP

用户可以选择屏蔽 1000 以下的所有端口（0、23、68、502 这四个端口保留,依然可以使用）。具体内容参考指令手册中的 IK 指令

与多个设备通讯

DMC-B140-M 能够支持多主多从通讯。“主”可以是多台 PC 机，发送命令给控制器；典型的“从”是外围 I/O 装置，从控制器接收命令。

注意：“主”等效于互联网的客户端（Client），“从”等效于互联网服务器（server）。

Ethernet 句柄（handle）是在设备内的通讯资源。DMC-B140-M 同时最多能够有 4 个 Ethernet 句柄开放。使用 TCP/IP 时，每个与 DMC-B140-M 通讯的设备都要占用一个句柄。如使用 UDP/IP，所有客户端共用一个句柄，但每一服务器占用一个单独的句柄（Pings 和 ARP 不占用句柄）。如果所有 4 个句柄均被占用，而第 5 个主设备试图接入，就会发送“reset packet”，在其应用程序中产生相应的报警。

注意：有许多方法对控制器进行复位：硬件复位（按下复位按钮或对控制器断电）及软件复位（通过 Ethernet 或 RS232 发送 RS 指令）。唯一不会使控制器断开网络连接的复位方法是由 Ethernet 发送 RS 指令进行软件复位。

当 Galil 控制器在通讯中作为客户端使用时，用 IH 命令来分配句柄并指定对方的 IP 地址。IP 地址按 4 字节数，中间用逗号分开输入，或者输入带符号 32 位数。可以指定端口（port）号，如果不指定，其缺省值为 1000。连接的同时指定使用的通讯协议（TCP/IP 或 UDP/IP）。否则，控制器就不会连接到从设备。例如：IHB=151, 25, 255, 9<179>2，这样就会打开句柄#2，并连接到 IP 地址 151, 25, 255, 9 的设备，端口号为 179，使用 TCP/IP 协议。

第五章 指令与程序

说明

在了解了运动控制器系统的构成、硬件连接、通讯之后，本章简单说明 DMC-B140-M 控制器指令及程序应用基础知识，以方便用户对以后各章节的理解。对于全部指令的列表及说明，请参见《指令手册》。

DMC-B140-M 为用户提供了 100 多条指令用于运动控制和参数设置。这些指令涉及到初始化、查询状态、运动控制等方面。

指令由 ASCII 字符构成，绝大部分指令本身由两个大写字母组成。这两个字母通常与相应英文单词相对应，便于用户记忆。例如：指令 BG 指开始运动（Begin），ST 指停止运动（Stop）。

用户能够随时利用通讯口向 DMC-B140-M 控制器发送指令，控制器收到指令后立即执行。也可以将整个一组指令作为程序下载到 DMC-B140-M 的存储器中，在需要的时候执行。大部分指令即可以从通讯接口发送执行，也可以在程序中执行，但有些指令只能用于立即执行的方式，而有些指令则只能在程序中使用。

指令从功能区分，可以分为动作指令、查询指令和程序流程控制指令。一个完整的指令通常由指令本身和指令参数组成，参数可能是数字或字符。一些指令本身不需要参数。需要参数的指令在具体的应用中也可以省略部分或全部参数。

指令格式

执行指令

需要制器执行指令时，只要输入代表指令本身的两个 ASCII 大写字母，后跟可能需要的参数。在指令本身与和参数之间可以有一个空格。最后用回车来结束指令输入。根据不同的指令内容，控制器可能开始某一动作，设置或查询某种状态参数。

例如，指令

PR 4000<Enter> 相对位移距离设置

PR 是用于指定相对位移量的指令，4000 位移的距离，是以发送脉冲数为单位表示。<Enter>表示回车键，由这样一个字符表示该条指令结束（以下不再明确标明<Enter> 的输入）。PR 和 4000 之间的空格可选。

一个 PR 指令可以最多有 4 个参数，分别指定 A, B, C, D 轴的位移距离。对于这样数值类型的参数，参数之间必须用逗号“,”分开，参数也必须按照正确的顺序输入。对于 PR 指令来说，紧跟在指令后的第一个参数，表示 A 轴的位移距离；跟在第 1 个逗号后的参数，表示 B 轴的移动距离；跟第 2 个逗号后的，表示 C 轴的移动距离；最后一个参数，表示 D 轴的移动距离。

如果某个参数不需要——例如，该轴不需要移动，也就不需要指定移动距离——可省略该参数，但如果在此参数后还有其它的参数，则相应的逗号不能省略。

大部分使用数值参数的指令可以将问号“?”作为参数使用。这样使用表示对控制器当前值的查询。

请看以下执行 PR 指令的例子：

PR1000	只对 A 轴指定 1000，后面的其它参数及逗号均省略
PR, 2000	只对 B 轴指定 2000，参数前面的逗号不能省略
PR, , 3000	只对 C 轴指定 3000，参数前面的两个逗号不能省略
PR, , , 4000	只对 D 轴指定 4000，参数前面的三个逗号不能省略
PR2000, 4000, 6000, 8000	对 A, B, C, D 轴指定位移值
PR, 8000, , 9000	只对 B, D 轴指定位移值
PR?, ?, ?, ?	查询 A, B, C, D 轴目前设置的位移值
PR, ?	查询 B 轴目前设置的位移值

对于类似 PR 这样设置各轴参数的指令，如果只需要一个参数，还有另外一种用法，直接在指令后输入对应轴名称，在跟上等号“=”和参数值。

PRA=1000	指定 A 轴相对位移为 1000
PRD=?	查询 D 轴目前设置的位移值

控制器的一些指令是以数值作为参数，而是以字符作为参数。一些与运动相关的指令，经常是以轴名称作为参数，例如指令：

BGA	A 轴开始按照之前设置的方式运动
-----	------------------

对于这一类型的指令，即使有多个参数，参数之间不需要任何字符来分割，参数的顺序也不重要。如果省略所有参数，则等效于输入所有参数。

BGA	A 轴开始运动
BG B	B 轴开始运动
BGABCD	四轴均开始运动
BGDB	B 轴和 D 轴开始运动
BG	四轴均开始运动

也有一些指令，参数本身与运动完全无关，例如，流程控制指令中最基本的延时指令 WT 以及用于输出信息的 MG 指令。WT 指令只能用于程序之中，不能单独执行。WT 指令只有一个参数，就是延迟时间，时间单位毫秒。下面的指令就可以使程序的执行暂停 1 秒：

WT1000

最简单的 MG 指令只需要一个参数，就是所输出的内容，以下是几个 MG 指令的例子

MG"HELLO"	控制器输出文本 “HELLO”
MG 10*100	控制器输出 10*100 的计算结果 “1000”

也有一些指令不需要任何参数，例如指令 BN，作用是将当前所有设置保存到控制器。

控制器对指令的回应

对于正确的指令，控制器会执行指令，并从接收到指令的通讯口返回信息。如果是查询类型的指令，所返回的信息包括所查询的内容，而后是一个冒号“:”（3AH），如果是非查询性的指令，则只回应冒号。

如果指令本身有错误，如指令拼写错误、参数不合逻辑或超出允许范围等等，控制器会回应一个问号“?”（3FH）。TC 指令可以查询出错原因的编号及说明，详细内容请参考《指令手册》中 TC 指令的说明。

如用户使用 Galil 的通讯程序，如 GalilTools，当输入的指令出现错误时，通讯程序本身会自动发送一个 TC1 指令查询具体的错误原因。

操作数

操作数是指令的另外一种应用形式。大部分指令都支持这种应用形式，还有一些指令只能作为操作数来应用。操作数可以看作是一种特殊的系统变量，它对应控制器的状态值。在语法上，操作数本身相当于数值，可以用做其它指令、函数的参数，或是参与数值运算。

指令用于操作数形式，通常是在指令前加一个下横线“_”（5FH）。多数指令的操作数形式代表的意义与其执行时的作用相关，例如指令

SPA=1000 设置 A 轴运动速度为 1000（脉冲频率为 1000Hz）

而操作数

_SPA

就是对应当前的 A 轴速度设置值。也可以使用这样的指令，使 B 轴的设置速度与 A 轴相同

SPB=_SPA

具体哪些指令支持操作数的应用形式，以及操作数对应的意义，请参阅《指令手册》。

程序基础

指令可以组合为程序，存储在 DMC-B140-M 控制器上，在用户需要的时候执行。关于程序编写的全部相关内容，请看第七章，这里只介绍一些基础知识，以便读者能够理解下一章中的内容。

程序的组成

程序由语句行组成，一个语句行中可以包含一个或多个指令，指令之间用分号“;”分开。语句行中最后一条指令末尾不需要分号。每个语句行都是以回车符结尾。语句行的长度受控制器的存储空间限制，对于 DMC-B140-M 控制器而言，普通语句行最大长度为 40 字符。

不同语句行是按照行号的顺序执行，语句行内的指令也是按照先后顺序执行。行号本身不是语句的一部分，用户在输入、编辑程序时也不需要输入。语句行的总数同样受控制器的存储空间限制。对于 DMC-B140-M 控制器而言，提供的用户程序存储空间可以容纳 450 个语句行。

除了需要执行的指令外，程序通常还会包含标号与一个结束指令 EN。

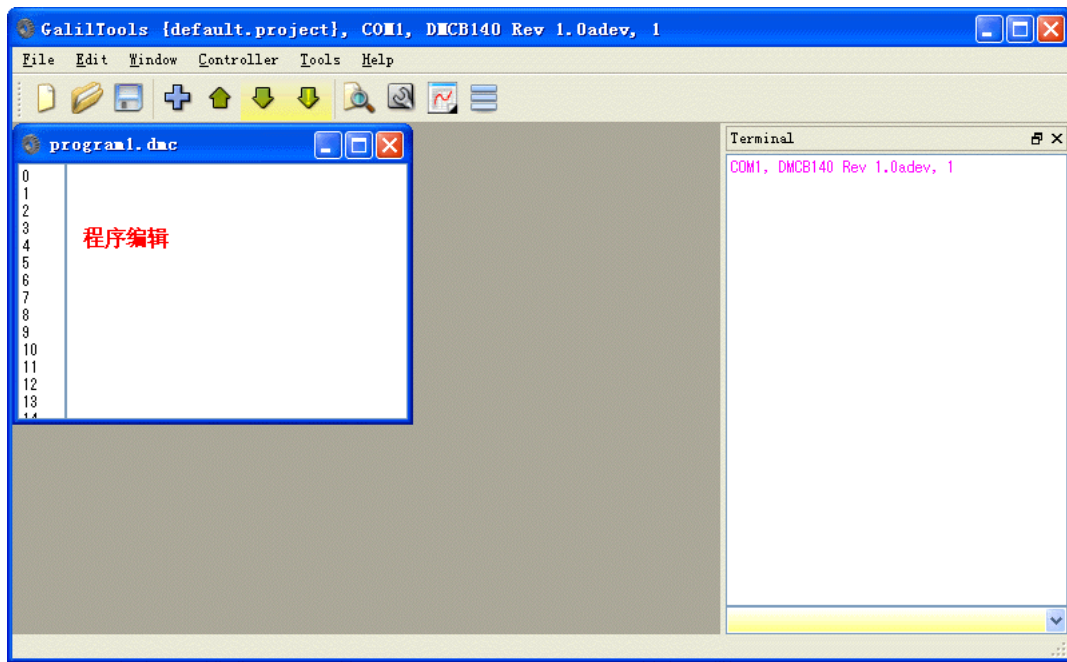
标号不是必须的，语句行本身的行号可以起到标号的作用。但适当的使用标号会增加程序的可读性，而且便于对程序进行编辑。

标号以符号“#”开头，之后是标号的名称。标号名称以字母开头，后面可以是字母或数字。标号名称总长度不能超过 7 个字符。一个语句行只能包含一个标号，而且标号必须在语句行的开头。对于 DMC-B140-M 控制器而言，整个程序中最多可以包含 62 个不同的标号。

EN 指令是表示程序结束的指令，当执行到 EN 指令时，程序就会停止，不再执行后面的指令。

程序的输入与编辑

在 GalilTools 软件中，点击工具栏中的  图标，就可以出现一个新的程序编辑窗口，用户可以在该窗口内输入和编辑程序。如果用户有之前保存的程序文件，也可以直接打开文件进行编辑。



程序的下载与运行

程序必须下载到控制器中才可以执行。在 GalilTools 软件中，需要将编辑好的程序下载执行，只要点击工具栏中的 。如果只是下载程序，暂不执行，则选择 。在需要时，输入 XQ 指令就可以执行已经下载到控制器中的程序。

控制器已下载程序的查看

要查看已经下载到控制器中的程序，有两种方式。一种方式是输入指令 LS。这样，就会在控制器输出窗口中显示出当前控制器中程序的全部内容。

另一种方式是将控制器中的程序上载到新的程序编辑窗口中。工具栏中的  可实现此功能。

程序的保存

将程序编辑窗口中的程序保存在 PC 上，只要先选中需要保存的窗口，点击工具栏中的 ，或使用菜单中的 save 或 save as 就可以了。

输入指令 BP 可以将下载到控制器中的程序永久保存在控制中。除非保存新的程序将其覆盖，或者对控制器实行主复位，否则被保存的程序不会丢失。

第六章 控制

概述

DMC-B140 作为运动控制器，最基本的就是控制功能。DMC-B140 提供了简单的 I/O 控制以及多种运动方式，它包括定位、点动、插补运动、电子凸轮运动、电子齿轮等。用户可以非常方便的利用这些现有的运动方式根据控制需求编写出应用程序。另外，GALIL 还提供了一些与运动状态关系密切的特殊功能。本章将对这些控制器的这些控制功能进行介绍。

I/O

I/O 控制是最基本的控制功能，也是运动控制卡除了运动控制以外必须具备的功能。DMC-B140-M 控制器除了一些与运动控制密切相关的输入/输出信号外，还提供了一些通用的 I/O 信号。通用信号包括 8 路数字输入与 4 路数字输出（其中通用输入 8 默认作为控制器的急停输入使用）。I/O 控制最基本的内容就是读出输入的状态与设置输出的状态。

查询指令 TI 可以读出 8 路通用输入的状态，TI 指令的反馈值是一个在 0~255 之间的数据，将此数据转换为 8 位二进制的数值后，每一位依次表示一路输入的状态，最低位表示通用输入 1，最高位表示通用输入 8。DMC-B140-M 控制器上通用输入信号截止状态下，对应位的数值为 1，当某路输入信号导通时，对应的数值为 0。当 DMC-B140-M 的 I/O 接口未连接时，如在 GalilTools 软件中输入 TI 指令，则反馈数值 255（如用户改变了 AB 跳线设置，则反馈 127）

TI 指令的操作数 TI 与指令反馈值意义相同。指令 TI 与指令 MG_TI 的作用是一样的。

用于设置输出状态的指令有 4 个，分别是 SB、CB、OB、OP。

在出厂默认状态下，每路输出信号的正向引脚与负向引脚之间是不导通的（作为电气保护措施，所有的输出在任何状态下，反向均导通，参见第三章）。SB 指令是设置输出位（set bit）导通。SB 指令的格式为：

SBn n 表示要设置的输出号，为 1~4 之间的整数

指令 CB 用来清除输出位（clear bit），将输出恢复到截止状态。CB 指令的使用方式与 SB 指令是完全一样的。

指令 OB 也是用来设置单路输出状态的指令，格式为：

OBn, m n 表示要设置的输出号，为 1~4 之间的整数，m 为设置的状态 0 或 1

参数 m 的值为 0，则设置输出为截止状态；值不为 0，则设置输出为导通状态。实际应用中，参数 m 通常为一个运算表达式。关于数值与逻辑运算的内容，将在下一章介绍，下面只举一个简单的例子：

OB4, _BGA

这样，第 4 路输出的状态，就取决与指令执行时 A 轴的运动状态：如 A 轴在运动中，则第 4 路输出导通；如 A 轴静止，则第 4 路输出截止。

指令 OP 可以同时设置所有通用数字输出的状态，指令格式为：

SBn n 表示要设置的状态值，为 0~15 之间的整数

参数 n 的值转换为 4 位二进制数字，则每一位的数值依次对应一路输出的状态，最低位对第一路输出，最高位对应第 4 路输出。数值为 1 表示将该路输出设置为导通状态；数值为 0 表示将该路输出设置为截止状态。例如，需要设置输出 1、2、4 导通，3 截止，则 OP 指令的参数值为二进制的 1011，对应十进制数值 11，则可以输入指令：

OP11

指令 OP 的操作数_OP 表示当前通用输出的状态，其意义与 OP 指令的参数相同。

当改变了输出的状态后，如执行了 BN 指令，那么当前的状态将被控制器记忆，当控制器下次开机或复位时，将在完成自检后将所有输出恢复到这一状态。

I/O 控制指令汇总

指令	说明
TI	查询全部通用输入状态
CB	设置输出位截止
SB	设置输出位导通
OB	依据参数值设置输出位
OP	设置全部通用输出
_TI	当前全部通用输入状态
_OP	当前全部通用输出状态

独立定位

所谓独立轴定位控制，就是说各轴之间的运动独立，与其它轴的状态无关，各轴按照程序中所规定的运动轨迹完成各自的定位控制。用户为各轴指定目标绝对位置（PA）或相对位置（PR），速度（SP），加速度（AC），减速度（DC），让其开始运动（BG），控制器就自动产生相应的速度和位置规划，完成定位运动。

注意：当控制器规划的运动完成时，由于电机及负载惯性，实际电机运动并不一定立即完成，但这并不影响发送下一条运动指令。

控制器可以对所有各轴同时或分别发送 BG 指令，需要用 A，B，C，D 来选择所要运动的轴。当未指定轴号时，就意味着使所有轴开始运动。在运动期间，可以随时改变运动速度（SP）和加速度（AC），不过，直到运动完成后，才可以改变减速度（DC）和位置值（PR 或 PA）。在到达终点位置之前，可以随时发送停止指令（ST）使电机减速停止。

只要附加运动与正在执行的运动方向相同，在运动期间就可以指定增量位置（IP），此时用户只指定新的位置增量 n ，新的目标位置就等于原有目标位置+增量 n 。一旦接收到 IP 指令，就会重新规划运动轨迹，使运动达到新的终点位置。IP 指令不需要 BG。

注意：如果电机不在运动中，IP 指令就等效于 PR 和 BG 指令的组合。

独立定位指令汇总

指令	说明
PR	指定相对位置
PA	指定绝对位置

SP	指定运动速度
AC	指定加速度
DC	指定减速度
BG	开始运动
ST	停止运动
IP	增量位置
IT	速度平滑的时间常数
AM	等待运动规划完成
MC	等待实际脉冲发送完成*
_PRx	当前“x”轴的增量位置值
_PAx	若“x”轴不在运动中，读取当前置值，而如果处于运动状态，则读取之前停止位置值
_SPx	当前“x”轴的速度设置
_ACx	当前“x”轴的加速度设置

(*由于控制器对输出脉冲信号进行平滑处理，所以实际脉冲信号发送完成会比运动规划结束的时间滞后几个毫秒，请参看 KS 指令)

举例：

1. 绝对位置运动

指令

PA 10000, 20000	指定 A, B 轴绝对位置
AC1000000, 1000000	指定 A, B 轴加速度
DC 1000000, 1000000	指定 A, B 轴减速度
SP 50000, 30000	指定 A, B 轴速度
BG AB	开始运动

2. 多轴顺序运动

要求运动参数：

A 轴：	位置	2000 脉冲
	速度	15000 脉冲/sec
	加速度	500000 脉冲/sec ²
B 轴：	位置	1500 脉冲
	速度	10000 脉冲/sec
	加速度	500000 脉冲/sec ²
C 轴：	位置	100 脉冲
	速度	5000 脉冲 / sec
	加速度	500000 脉冲 / sec ²

此例要求 A, B, C 轴做相对位置运动，各轴运动间隔为 20msec，图 6.1 表示轴的速度轮廓。此例的程序如下：

指令	说明
#A	程序标号
PR2000, 1500, 100	A, B, C 轴运动距离分别为 2000, 1500, 100 计数单位
SP15000, 10000, 5000	A, B, C 轴运动速度分别为 15000, 10000, 5000 计数单位 /sec

AC500000, 500000, 500000	各轴加速度为 500000 计数单位/sec ²
DC500000, 500000, 500000	各轴减速度为 500000 计数单位/sec ²
BGA	开始 A 轴运动
WT20	等待 20msec
BGB	开始 B 轴运动
WT20	等待 20msec
BGC	开始 C 轴运动
EN	程序结束

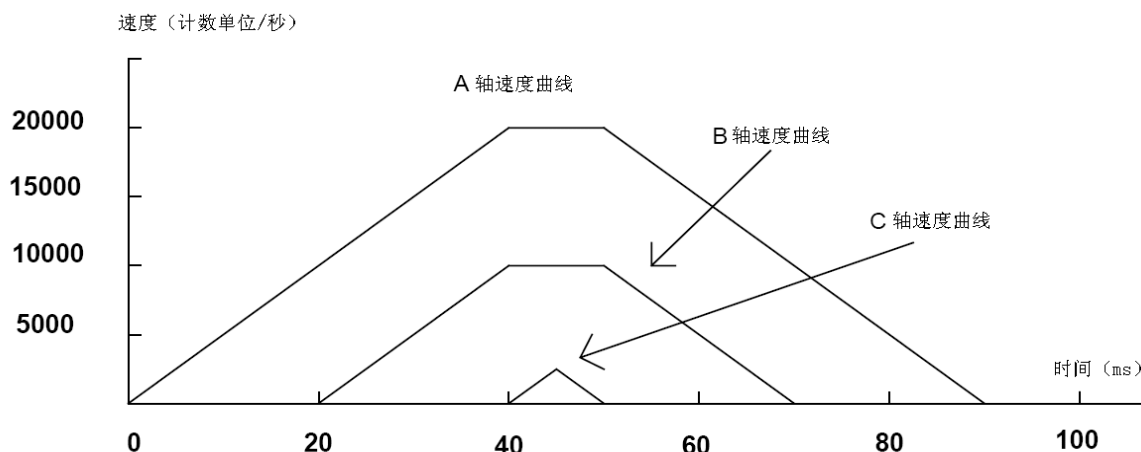


图 6.1 ABC 速度轮廓

图注：A、B 两轴有一个“梯形”速度曲线，而 C 轴有一个“三角形”速度曲线。AB 轴加速度到指定速度，以恒速运动。然后减速到达位置指令所指定的终点。C 轴加速，但在到达指定速度之前，就必须减速以使该轴停在指定的位置。所有三个轴均有相同的加，减速度，因此，三个速度轮廓的上升，下降的斜率是相同的。

JOG 运动

JOG 运动方式非常灵活，这是因为在运动期间，不但可以随时变更运动速度、加速度，而且可以改变运动方向。用户对各轴规定 JOG 速度 (JG)，加速度 (AC) 和减速度 (DC)。运动方向由 JG 参数的符号来决定：当用 BG 使运动开始时，电机加速到达所设速度，并以此速度连续运转，直到有新的速度给定或发送 ST 指令。如果在运动期间改变 JOG 速度，控制器就会使电机加速或减速达到新速度。

通过使用 IP 指令能够使电机位置发生瞬时改变，一接收到此指令，控制器就指令电机到达给定增量加上当前位置的新目标位置，此指令对于在电机运行过程中，想使两个电机的位置同步是非常有用的。

注意：IP 指令的参数不能过大，如果要求的瞬时速度超过控制器的极限速度 (3MHz) 则控制器会停止输出脉冲。

JOG 运动指令汇总

指令	说明
----	----

AC	指定加速度
DC	指定减速度
BG	开始运动
ST	停止运动
IP	指定瞬时运动增量
IT	使加、减速过程平滑的时间常数
JG	指定 JOG 速度和方向
_SPx	当前 “x” 轴的速度设置
_ACx	当前 “x” 轴的加速度设置
_DCx	当前 “x” 轴的减速度设置
_TVx	当前 “x” 轴的编码器反馈速度
_BGx	当前 “x” 轴是否在运动*

(*如果 JG 指令给定速度为 0，此时虽然控制器不会发出脉冲，但仍然认为此轴处于运动状态。)

举例：

使 A 轴以 50000 脉冲/sec 速度做 JOG 运动，在 A 轴电机到达 JOG 速度后，使 C 轴以 25000 脉冲/sec 速度做反方向运动。

指令	说明
#A	程序标号
AC20000, , 20000	指定 A, C 轴加速度为 20000 计数单位 / sec^2
DC20000, , 20000	指定 A, C 轴减速度为 20000 计数单位 / sec^2
JG50000, , -25000	指定 A, C 轴 JOG 速度和方向
BGA	开始 A 轴运动
ASA	等待 A 轴速度到达
BGC	开始 C 轴运动
EN	程序结束

位置跟随

在独立轴定位控制方式下，在运动过程中可以改变运动速度、加速度但不能任意改变目标位置。而在一些应用中，电机在运动中跟踪一个随机位置，无论一个轴是否在运动，都要即刻改变最终位置。位置跟随模式可以满足这样的要求。位置跟随模式同样需要指定速度 (SP)、加速度 (AC) 和减速度 (DC) 指令。启动位置跟随模式后，随时可以指定目标位置 (PA)。不需要 BG 指令来开始运动，指定位置后，控制器就会根据当前的位置、速度和目标位置来生成或修改运动规划。

启动位置跟随所用的指令为：

PT n, n, n, n。

当 n 为 1 时，启动对应轴的位置跟随模式；当 n 为 0 时，关闭对应轴的位置跟随模式。

在位置跟随模式启动后，控制器即认为该轴处于运动状态，直至模式关闭。

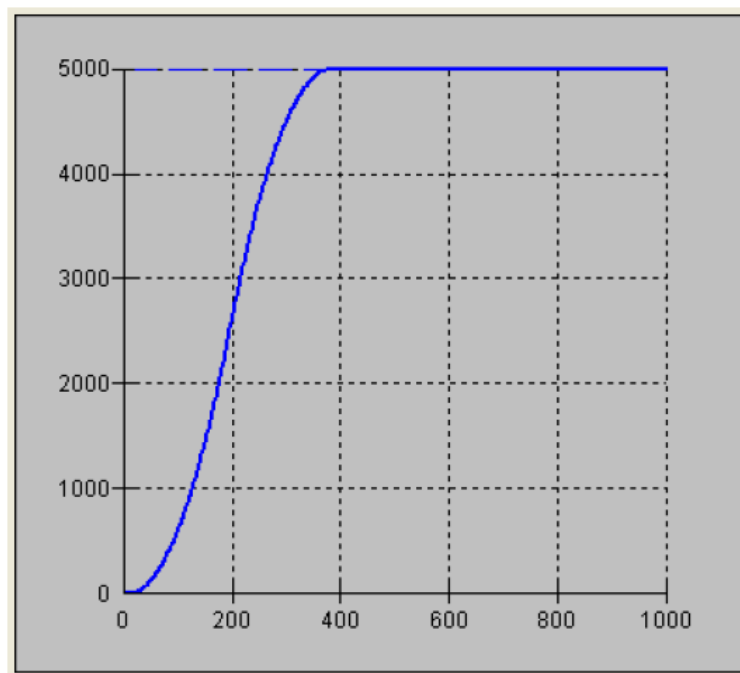
位置跟随指令汇总

指令	说明
PT	开启/关闭位置跟随功能
PA	指定目标位置
SP	指定运动速度
AC	指定加速度
DC	指定减速度
_PTx	当前“x”轴位置跟随模式的状态
_PAx	若“x”轴不在运动中，读取当前规划位置值，而如果处于运动状态，则读取之前停止位置的规划位置
_SPx	当前“x”轴的速度设置
_ACx	当前“x”轴的加速度设置
_DCx	当前“x”轴的减速度设置

举例：

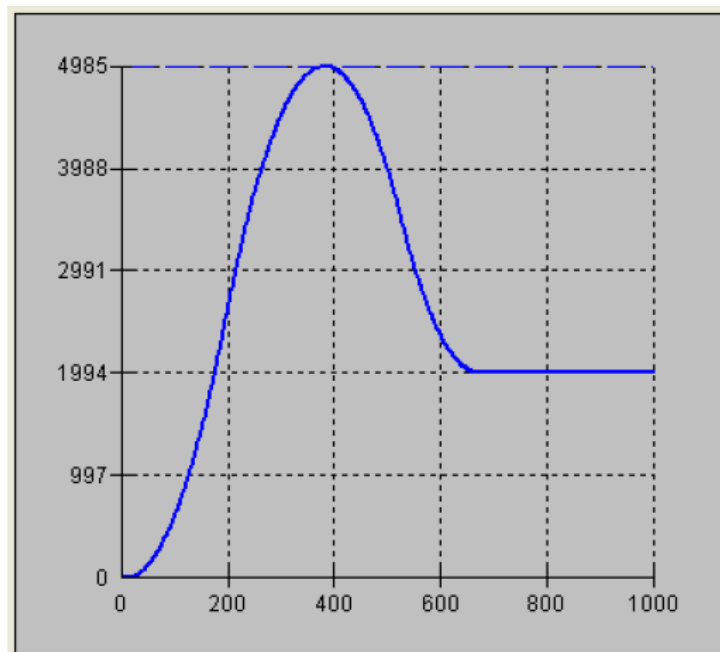
指令
PT 1 使用位置跟随
AC 150000 指定 A
DC 150000 指定 A
SP 50000 指定 A
PA 5000

上面程序的运动过程如下图：

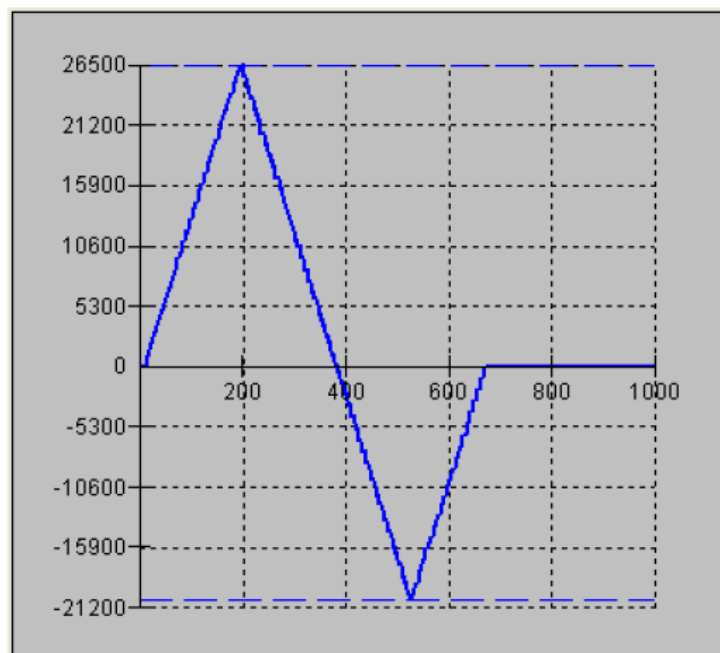


a.位置-时间（ms）曲线

如果在到达 5000 之前，发送 PA 2000 指令，则可以得到如下运动过程：

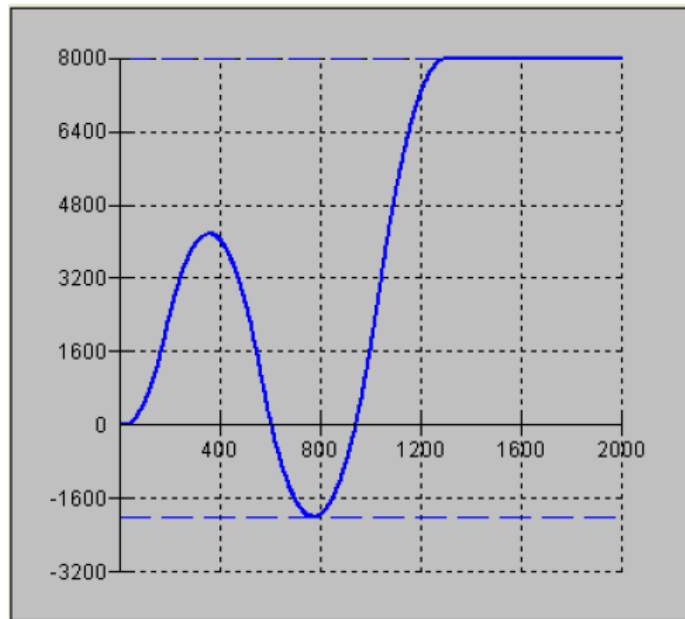


b.位置-时间（ms）曲线

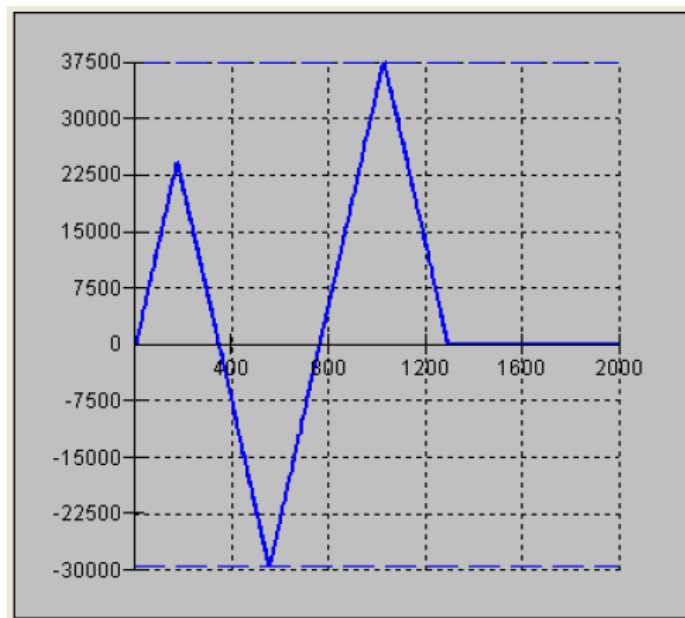


c.速度-时间（ms）曲线

如果在运动中，先将目标位置改为-2000，而后再改为 8000，则运动过程下：



d.位置-时间（ms）曲线



e.速度-时间（ms）曲线

多维直线插补

控制器提供最多 4 轴直线插补功能。在直线插补方式中，各轴沿指定的轨迹以矢量速度，加速度，减速度联动；运动轨迹根据各轴的增量距离来产生。在连续插补运动中，可以无限给定增量线段，对总的运动长度没有限制。

LM 指令为插补功能选择直线插补方式和插补轴。例如，LMBC 只选择 B 轴和 C 轴为直线插补。使用直线插补方式时，如果不改变直线插补轴，那么只需要指定一次 LM 指令。

插补矢量轴

对于直线插补方式或下面描述的平面矢量方式而言，控制器插补的矢量运动做为一个特殊的轴 S 来描述。开始插补运动，使用指令：

指定直线段

用指令 **LI n, n, n, n** 来为各轴指定增量运动距离, 每个 **LI** 指令向缓冲区中写入一段插补路径。控制器的插补路径缓冲区可以容纳 31 段路径, 也就是说, 在 **BGS** 令之前, 可以写入 31 个 **LI** 指令。开始运动后, 电机按照缓冲区中的逐条给定的路径运动, 缓冲区的空间也逐条释放。缓冲区中有空间后, 还可以继续向缓冲区内添加新的插补路径段。

在运动开始之前, 可以用清空缓冲区的指令 **CS** 来删除存储在缓冲区中的内容。要想停止运动, 可以使用指令 **STS** 或 **AB**。**ST** 指令使电机减速停止, 而指令 **AB** 使电机即刻停止并使控制器上的用户程序终止执行, (指令 **AB1** 只使运动终止, 不会中断用户程序)。

必须使用直线插补路径结束指令 **LE** 来说明直线插补运动曲线完成。此指令告诉控制器最后一条 **LI** 指令就是插补运动的终点, 不再接受新的 **LI** 指令 (除非使用 **LM** 指令重新定义新的插补曲线)。如果未给定 **LE** 指令, 就必须用终止指令 **AB1** 来终止运动。

要使运动连续平稳, 用户需要保证缓冲区中保持足够的路径段。如果控制器未接收到新的 **LI** 指令或 **LE** 指令, 在最后一段路径完成之后, 控制器就会立即停止运动, 也没有可控制的减速度。**LM?**或**_LM** 读取能够发送到缓冲区可用空间。返回值为 31 意味着缓冲区是空的, 还可以写入 31 段路径。若返回值为 0, 则意味着缓冲区是满的, 不能再执行 **LI** 指令了。只要缓冲区未滿, 就能够继续执行 **LI** 指令。

指令 **_CS** 读取路径段计数器值, 当按照给定插补路径运动时, 每完成一段路径, **_CS** 加 1, 此计数器值以 0 开始计数。此功能使得主计算机能够确定正在处理哪段路径。

附加指令

用指令 **VS_n**, **VA_n** 和 **VD_n** 来指定矢量速度, 矢量加速度和减速度。控制器以 **LM** 指令中所指定的轴为基础计算矢量速度。并默认所有轴之间为正交关系。例如, **LMABC** 指定 A, B, C 轴为直线插补轴, 则矢量速度用下式进行计算:

$$VS^2 = AS^2 + BS^2 + CS^2$$

其中: **AS**, **BS**, **CS** 分别为 A, B, C 轴的速度。

控制器总是使用由 **LM** 所指定的轴定义来计算速度, 而不是 **LI**。

指令 **AV_n** 是“矢量运动完成后”条件启动指令, 它使程序暂停执行, 直到矢量距离 **n** 到达之后, 才重新启动执行后续程序。

对各插补线段指定矢量速度

指令 **VS** 具有立即效应, 因此, 必须在需要时刻给定。在某些应用中, 例如 **CNC**, 需要对不同的运动线段指定各种不同的速度; 这可以用两个参数来进行 (<**n** 和 >**m**)。

例如: **LI a, b, c, d<n>m**

第一个参数 <**n** 等效于在给定线段的起点指定 **VS_n**, 在其约束下, 使运动朝着新的速度变化, 如果将 **n** 设置为 -1, 则表示速度将在实际运动时由指令 **VV** 设定。第二个参数 >**m**, 指定矢量速度在线段终点要达到的值。注意: 在给定的 **VA**, **VD** 值条件下, 参数 >**m** 可能在给定的线段内或在前一线段期间开始减速, 满足最终速度要求。

不过, 请注意: 控制器一次只有一个 >**m** 指令起作用, 因此, 一个参数可能会被另一个屏蔽。例如, 如果参数 >100000 由 >5000 跟随其后, 控制器会优先满足 >100000 的要求, 如果减速距离不足, 控制器就不会在指定线段的终点将速度降到 5000。

作为一个例子, 看看以下程序。

指令	说明
#ALT	程序标号
DP, 0, 0	定义 B, C 轴位置为 0
LM BC	定义 B, C 轴之间为直线插补方式
LI, 4000, 0<4000>1000	指定第一段插补路径: B 轴走 4000, C 轴走 0, 矢量速度为 4000, 结束速度为 1000
LI, 1000, 1000<4000>1000	指定第二段插补路径: B 轴走 1000, C 轴走 1000, 矢量速度为 4000, 结束速度为 1000
LI, 0, 5000<4000>1000	指定第三段插补路径: B 轴走 0, C 走 5000, 矢量速度为 4000, 结束速度为 1000
LE	结束直线插补路径
BGS	开始运动
EN	程序结束

改变进给率

指令 VRn 使进给率 VS 以 0.0001 的分辨率在 0~10 之间成比例变化。此指令立即起作用并使 VS 按比例变化。VR 也适用于用“<”指定矢量速度时。此指令对于进给率修调功能非常有用。VR 不对加速度进行比例处理。例如, VR0.5 使 VS2000 的实际速度减小一半。

直线插补指令汇总

指令	说明
LM	指定直线插补轴
LI	指定相对于当前位置的增量距离, 并分配矢量速度
VS	指定矢量速度
VA	指定矢量加速度
VD	指定矢量减速度
VR	指定矢量速度比率
VV	矢量速度变量
BG	开始直线插补运动
CS	清除缓冲区中路径,
LE	直线插补路径结束
AM	等待运动完成
AV	在相对矢量距离到达之后的条件启动
_AV	读取已运动的相对矢量距离
_CS	插补路径段计数器——读取运动完成的插补路径段数, 以 0 开始
_LE	读取矢量长度(在 2147483647 之后复位)
_LM	读取控制器缓冲区中可用空间, 0 代表缓冲区已满, 31 表示缓冲区空
_VP	读取沿轨迹最后一个数据点的绝对坐标值

举例：

1) 直线插补运动

在本例中，要求 A，B 轴进行 90 度转向，为了在拐角附近减慢速度，我们使用 AV4000 条件启动指令，将速度减慢到 1000cts/sec，一旦电机到走过拐角，速度就增回到 4000cts/sec。

指令	说明
#LMOVE	程序标号
DP0, 0	定义 AB 轴位置为 0
LMAB	定义 AB 轴为直线插补关系
LI5000, 0	指定第一段插补路径
LI0, 5000	指定第二段插补路径
LE	直线插补路径结束
VS4000	指定矢量速度
BGS	开始运动
AV4000	设置条件启动，等待到达矢量距离 4000
VSI000	改变矢量速度
AV5000	设置条件启动，等待到达矢量距离 5000
VS4000	改变矢量速度
EN	程序结束

此例中的第一段路径，其中，A 轴向 5000 的位置运动。当运动到 3000 的位置时，各操作数的值应为：_AV=3000，_CS=0，_VPA=0，_VPB=0。

当运动到第二条线段中，B 轴坐标为 2000 的位置时，各操作数的值应为：_AV=7000；_CS=1；_VPA=5000；_VPB=0。

2) 直线运动

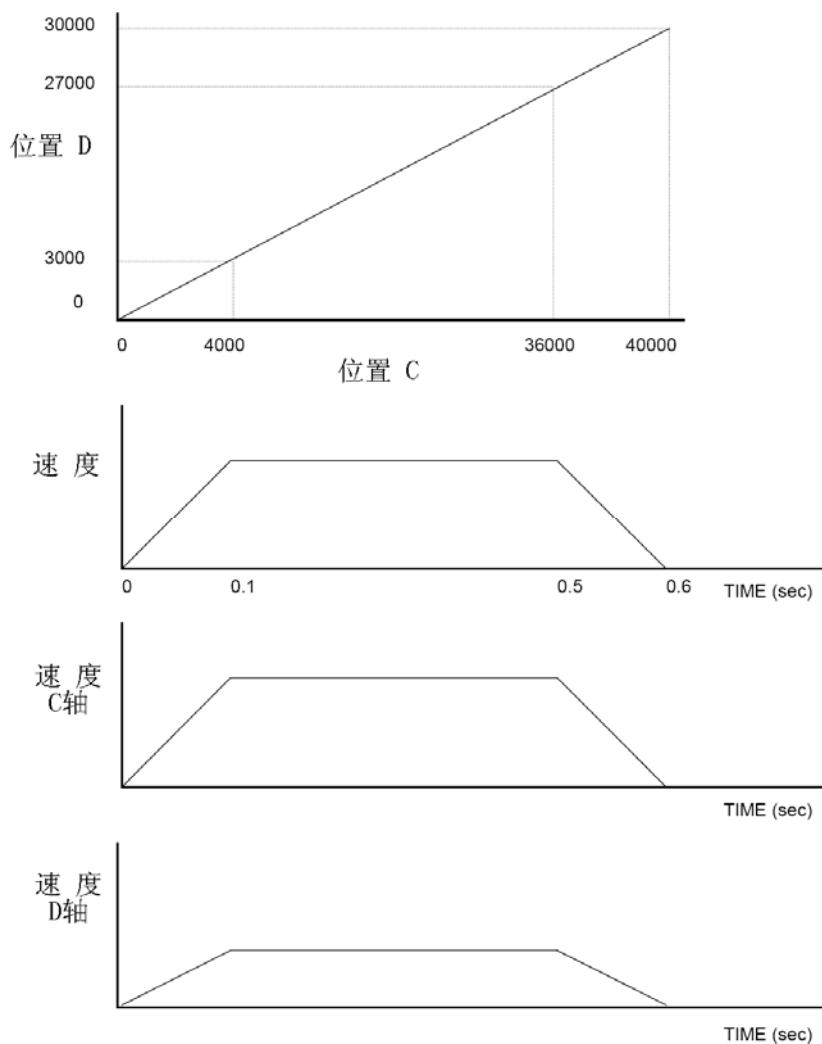
在 CD 平面做联动控制直线插补运动，以 100000 脉冲/sec 矢量速度和 100000 脉冲/sec² 的矢量加速度移动到坐标点 40000，30000。

指令	说明
LMCD	指定直线插补轴
LI, , 40000, 30000	指定路径
LE	直线插补路径结束
VSI00000	指定矢量速度
VAI000000	指定矢量加速度
VDI000000	指定矢量减速度
BGS	开始运动

注意：上述程序指定的矢量速度 VS，并不是实际轴速度 VC 和 VD，轴速度由下式求得：

$$VS = \sqrt{VC^2 + VD^2}$$

各轮廓曲线见图 6.2



平面直线和圆弧插补方式

控制器提供了平面内（二维）直线与圆弧插补功能，直线插补和圆弧插补可以混合编程。即使在直线和圆弧之间的过渡处，沿轨迹的运动也是以矢量插补速度连续进行。DMC-B140 执行所有直线、圆弧插补复杂运算，上位 PC 机可脱机去处理其他任务。而对于 3 维以上的圆弧插补，可以用高级语言（如：C / C++、VC 等），并调用 API 函数，在开放式平台上进行二次开发，以满足这方面的应用要求。

二维平面运动方式类似于多维直线插补方式。可以选择任意两个轴为联动轴构成直线和圆弧插补运动。

指令 VM m, n 其中 m 和 n 是两个联动轴。

注意：指令中的逗号（,）不是必须的，可以省略。例如，VMAB 选择 AB 轴为联动轴。

矢量路径

插补路径用两个指令进行描述，它们分别是：VP 为直线插补路径；CR 为圆弧插补路径。每个 VP 或 CR 指令都是一段插补路径，指令 VE 用来说明插补路径结束。在执行第一段路径前，控制器先将插补程序中所有运动轴的当前矢量位置定义为 0。

注意：局部 0 定义不影响绝对坐标系或以后的插补运动程序。

VP x, y 指令规定矢量运动终点相对于起点的坐标值。指令 CR t, q, d 以半径 r, 起始角 q 和弧角 d 来定义圆弧的弧长。起始角 q 约定为：0 对应于轴正方向，而对于 q 和 d, 逆时针方向旋转为正向，单位为角度（一周为 360 度）。

在开始运动前可指定 31 段 CR 或 VP 路径，用 BGS 来启动矢量运动程序，一旦开始运动，就可以继续加入路径段。必须用指令 VE 来结束路径的定义。此指令需要控制器减速停止在最后一插补运动之后。若未给定 VE 指令，就必须用 ABI 来中止插补控制程序执行。

在运动开始之前可以用 CS 指令来删除存储在缓冲区中的 VP 和 CR 指令。用指令 STS 或 ABI 来停止运动。STS 指令以规定的减速度使运动停止；ABI 立即使运动中止。

用户有必要在控制器的插补缓冲区中存入足够的矢量路径段，以确保连续运动。如果控制器未接收到新的路径段，且没有 VE 指令，控制器就会立即使运动停止在最后的路径处，也不会有可控制的减速度过程。LM?或 LM 可以读取缓冲区中的可用空间。返回值为 31 表示缓冲区是空的，此时可定义 31 段路径；0 表示缓冲区是满的，此时不能 VP 或 CR 指令定义新的路径段。只要缓冲区未满，就可以继续执行 VP 或 CR 指令。

用操作数_CS 来确认插补路径段计数器的值。

附加指令

指令 VS_n, VA_n 和 VD_n 用于设置矢量速度，加速度和减速度。

对各插补线段指定矢量速度

可以用即时指令 VS 指定矢量速度。也能将它用如下方式附加到路径段中：

VPa, b<n>m

CRr, θ, δ<n>m

第一个参数<n>, 等效于在运动到给定路径段起点加入指令 VS_n, 若无其他限制，它会使运动速度变为新指定的速度。如果将 n 设置为-1, 则表示速度将在实际运动时由指令 VV 设定。

第二个参数>m 规定了到达路径段终点处的矢量速度值。

注意：在给定 VA 和 VD 值条件下，参数>m 可能使运动在给定的路径段内或在前一路径段期间开始减速以满足最终的速度要求。

但是，请注意，控制器每次只能以一个>m 指令工作，因此，一个参数可以用另一个进行屏蔽；例如，如果参数.>100000 由>5000 跟随其后，且减速距离不够，那么就不满足第二个条件，此时，控制器会将速度减到 5000，但不会在指定位置达到。

改变进给率（进给率修调）

指令 VR_n 使进给率 VS 以 0.0001 分辨率在 0~10 之间进行比例修调。此指令立刻起作用并使 VS 成比例修正。VR 也适用于用“<”操作数指定矢量速度的场合。此功能对于像 CNC 那样要求有进给率修调功能的应用非常有用。VR 不对加，减速度做比例处理。例如，VR 0.5 使 VS2000 的实际速度减小一半。

分辨率修正

当使用二维插补方式时，控制器默认两轴的脉冲分辨率是一致的。如果实际情况与此不同，可以用指令 ES 对分辨率进行比例修正。ES 指令附带两个参数，分别代表用于矢量运动的两个输出脉冲的计数值。两个数的较小的一个作为比例基准。此指令对于椭圆加工非常有用，通常称此为椭圆缩放系数。详见《指令手册》ES 指令。

条件启动

AV_n 指令是在矢量运动完成后的条件启动，在执行程序中的下一条指令之前，等待相对矢量距离 n 到达。

平面插补控制指令汇总

指令	说明
VM	指定二维平面联动轴
VP	指定直线路径
CR	指定圆弧路径
VS	指定矢量速度
VV	矢量速度变量
VA	指定矢量加速度
VD	指定矢量减速度
VR	指定矢量速度比率
BG	开始运动
CS	清除缓冲区中路径，
VE	联动结束
AM	在矢量运动完成之后的条件启动
AV	在相对矢量距离到达之后的条件启动
ES	两轴定标比例系数
_AV	读取已运动的矢量距离
_CS	路径段计数器——读取运动结束的路径段数，以 0 开始
_VE	读取矢量长度(在 2147483647 之后复位)
_LM	读取控制器缓冲区中可用空间，0 代表缓冲区已满，31 表示缓冲区空
_VPx	读取沿轨迹最后一个数据点的绝对坐标值，（x: A, B, C, D）
_VM	当前实际矢量速度

举例

平面运动

运动路径如下图所示，进给率为 20000 脉冲 / sec，运动平面为 AB。

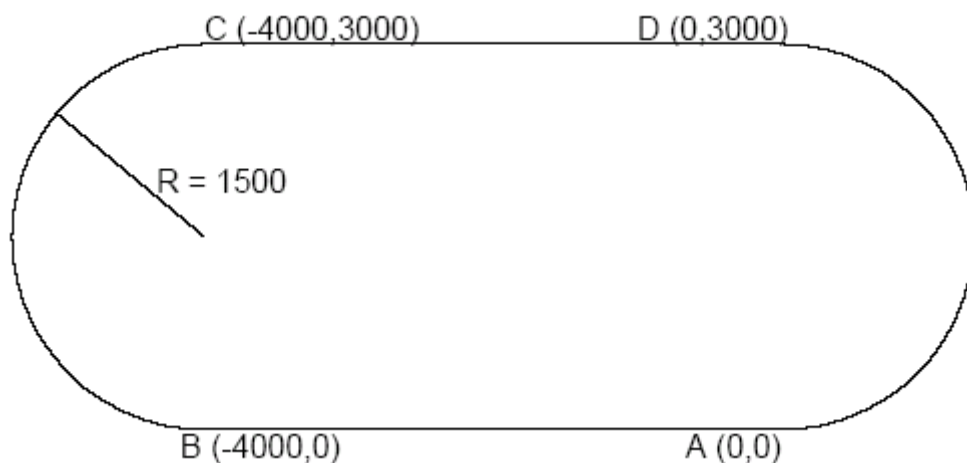


图 6.3-运动路径

指令

说明

VM AB	指定运动平面
VS20000	指定矢量速度
VA1000000	指定矢量加速度
VD1000000	指定矢量减速度
VP-4000, 0	线段 AB
CR1500, 270, -180	圆弧 BC
VP0, 3000	线段 CD
CR1500, 90, -180	圆弧 DA
VE	矢量路径结束
BGS	开始运动

此运动以 A 点开始，向点 B，C，D，A 运动，假设，运动到中途 AB 两点之间时，查询控制器。

_AV 的值是 2000

_CS 的值是 0

_VPA 和_VPB 是 A 点的绝对坐标值

假设在 C、D 两点中间再次查询：

_AV 的值是 $4000+1500\pi+2000=10,712$

_CS 的值是 2

_VPA 和_VPB 为 C 点的坐标值。

电子齿轮同步功能

此方式允许各个轴从电气上与其它的主轴实现同步运动。主动轴可以两个方向运动，而其它从动轴则以规定的同步比例跟随运动。各轴的同步比例可以不同，在运动期间可以随时改变同步比例。

指令 GA A, B, C, D 指定主动轴，GRa, b, c, d 指定从动轴的同步比例。同步比例可以是 ± 127.9999 之间的数，分辨率是 0.0001。

电子齿轮有两种方式，分别是标准电子齿轮同步和龙门同步方式。用指令 GM 使龙门同步方式有效。GR0, 0, 0, 0 关掉所有方式的电子齿轮同步控制。在标准方式中，电子齿轮的从动轴的限位开关或 ST 指令会使电子齿轮同步功能无效，但在龙门同步方式中，从动轴的限位开关无效，针对从动轴的 ST 指令无效。

指令 GMa, b, c, d 选择在龙门同步方式下要控制的轴。参数为 1 使用龙门同步方式，为 0 则使用标准电子齿轮功能。控制器出厂时默认为 0。

对于每个轴来说，可以有三个对应位置——规划位置、输出脉冲数代表的位置与编码器反馈信号代表的位置。前两个位置是基本相同的，只是在时间上，输出脉冲略滞后于规划位置。

可以指定主动轴的哪一个位置作为从动轴的同步基准。默认情况下，是使用主动轴的编码器反馈位置作为从动轴的同步基准。例如，GAB 表示电子齿轮轴是以 B 轴的编码器反馈位置来同步，A 轴会按照 B 轴的编码器反馈情况运动。用字母 C 来表明以主动轴的规划位置作为同步基准，指令 GACB 则表示电子齿轮轴是以 B 轴的规划位置来同步，A 轴会按照 B 轴的规划位置运动，也就是说，A、B 两轴输出的脉冲是一致的。用字母 D 来表明以主动轴的输出脉冲位置作为同步基准，指令 GADB 则表示电子齿轮轴是以 B 轴的输出脉冲位置来同步，也就是说，A 轴与 B 轴的输出脉冲基本一致，但 A 轴在时间上会滞后 B 轴一点（参见 KS 指令）。任何一个轴不可以本轴自身为主动轴。

GR 使指定的从轴与主动轴的同步比例系数。

主动轴不一定是 A、C、C、D 这四个独立的轴。插补运动中的 S 轴也可以作为主动轴。使用 S 轴作为主动轴时，是以规划的矢量运动距离作为同步基准，在 S 前不需要使用字母 C。例如，如果 A 轴和 B 轴形成圆弧运动，C 轴可以与矢量运动成比例运动。同样，如果 A、B、C 轴执行直线插补运动，就能使 D 轴与矢量运动成电子齿轮运动关系。

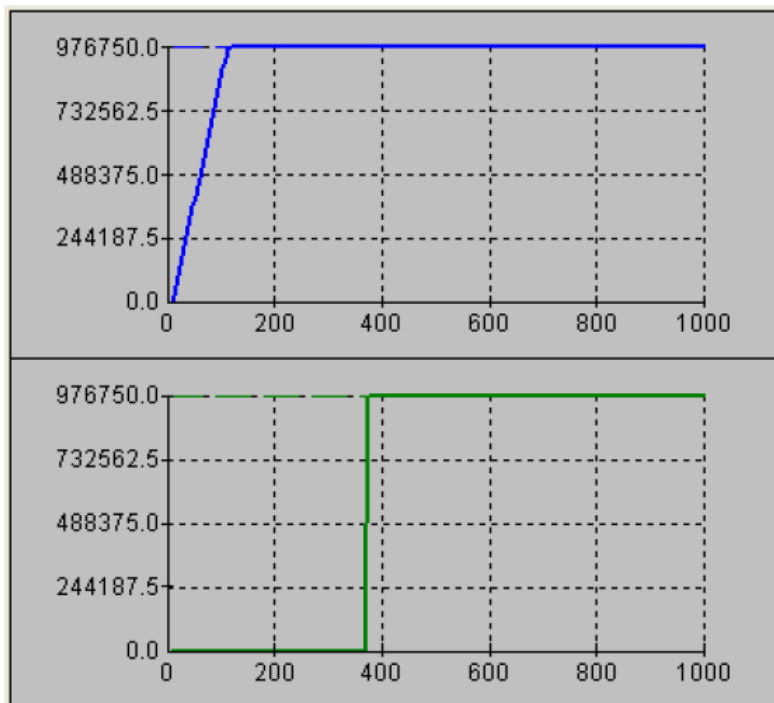
使用 S 轴作为主动轴时，则主动轴的运动方向永远被认定为正向。

电子齿轮功能也允许电子齿轮从轴电机执行除电子齿轮之外的独立轴运动或联动运动。则该轴的运动速度、距离为两种运动的叠加。例如，当电子齿轮轴电机以 1: 1 的比率跟随主动轴时，也可以用 PR、JG 指令或 VP、LI 指令使其超前一个附加的距离。

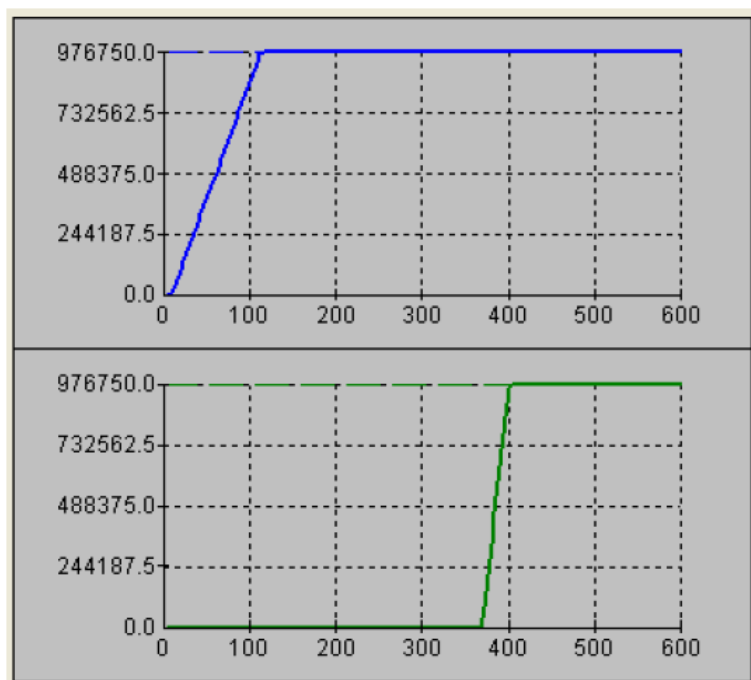
在一些应用中，需要在主轴高速运动中开/关电子齿轮功能，或者改变同步比例。例如，主轴在以 1,000,000cts/sec 的速度运行，从动轴实现 1: 1 同步。在开始同步的瞬间，从动轴的速度要瞬间达到 1,000,000cts/sec，电机要以最大加速度运行，这会给系统带来很大的冲击。在大多数应用中，这种冲击是不能接受的。而从动轴在一定的时间内逐步加速到达同步是更好的方式。Galil 控制器允许用户使用 GD 指令指定一段距离实现从动轴平缓加速过程。

例如，主轴同样是以 1,000,000cts/sec 的速度运行，同步比为 1: 1，但是，实现同步的距离为主轴的 30,000cts，这就可以很大的消除从动轴的机械冲击。下面两图是立即同步和逐渐同步过程的速度-时间曲线。

每个图中，下面的部分是从动轴，上面的部分是主动轴，在后一个图中，从动轴的机械震动会明显减轻。在加速过程完成前，两个轴并没有真正的完全同步，而是从动轴落后。如果从开始的一瞬间就需要精确的位置同步，则需要用在同步以外的指令对位置补偿。控制器可以用操作数_GP 跟踪这个相位延迟。



i.立即同步的速度-时间曲线



ii.逐渐同步的速度-时间曲线

用一个例子来说明_GPn 的作用。两个轴分别以 1.132 和-0.045 的比例与外部主轴同步，由于主轴在高速运动中，需要逐渐实现同步。C 轴作为外部主轴，A、B 两轴作为从动同步轴，实现同步的过渡距离为主轴的 6000cts。

MOC 关闭 C 轴，将其作为外部主轴
GAC, C C 轴的主编码器输入作为 A、B 的主轴
GD6000, 6000 指定加速距离为主轴的 C 轴编码器的 6000 个计数单位。
GR1.132, -0.045 指定同步比

在从动轴逐渐同步的过程中，主轴运动了 6000cts，从动轴分别只移动了大约 3396 脉冲、135 脉冲，相差的部分存储在操作数_GPn 中。如果需要精确的位置同步，用 IP 指令调整差值。

电子齿轮同步功能指令及操作数汇总

指令	说明
GA	电子齿轮主轴，
GD	电子齿轮同步的过渡距离
_GPn	改变同步比时，从动轴逐渐加速带来位置滞后量。
GR	为从动轴设置同步比，0 使所指定轴的电子齿轮无效
GM	1 设置为龙门同步方式，0 使龙门同步无效
MR	经过指定值时，使运动反向的条件启动，只能针对一个轴使用
MF	经过指定值时，使运动正向的条件启动，只能针对一个轴使用

举例

1) 简单主从运动

主动轴以 100000 脉冲/sec 速度移动 10000 脉冲，定义 B 轴为主动轴，A、C、D 轴与主动轴成电子齿轮运动，齿轮比分别为 5，-0.5，10。

指令	说明
GAB, , CB, CB	指定主动轴为 B 轴
GR5, , -0.5, 10	设置齿轮比
PR, 10000	指定 B 轴位移量
SP, 100000	指定 B 轴速度
BGB	开始运动

2) 电子齿轮

目的：以外轴主动轴速度的 1.132 倍和-0.045 倍运行两个电子齿轮电机，主动轴以 0~1800r/min（2000cts/rev 编码器）速度驱动主动轴。

方案：使用 DMC-B140 控制器，其中 C 轴为主动轴，A、B 轴为电子齿轮轴。

指令	说明
MOC	关断 C 轴使能，C 轴由外部控制，作为主动轴使用
GAC, C	指定 C 轴为 A、B 轴的主动轴
GR1.132, -.045	指定齿轮比

现在假设：A 轴的齿轮比要改变为 2，用以下指令来实现：

GR2	指定 A 轴齿轮比为 2
-----	--------------

3) 龙门同步方式

由 A、B 两轴在两侧驱动龙门架，对于两台电机之间的刚性连接而言，就需要龙门同步方式。A 轴是主动轴，B 轴是随动轴，要使 B 轴与 A 轴的规划位置同步，使用指令：

指令	说明
GACA	指定 A 轴规划位置为 B 轴的主动轴
GR 1	设定 B 轴的齿轮比为 1: 1
GM 1	设置龙门同步方式
PR 3000	指定 A 轴位置
BGA	启动 A 轴运动

在电子齿轮方式中，您也可以进行同步位置修正。假设，您需要使从动轴超前 10 脉冲，可以使用指令：

IP, 10	指定 B 轴增量位置位移量 10
--------	------------------

在这些条件下，此 IP 指令等效于：

PR, 10	指定 B 轴相对位移量 10
BGB	开始 B 轴运动

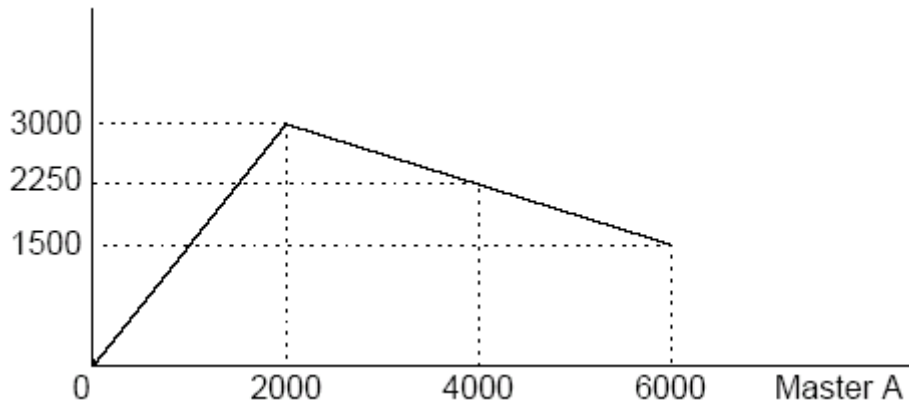
4) 使具有位置修正功能的两个输送带同步

指令	说明
GA, A	定义 A 轴为 B 轴的主动轴
GR, 2	设置 B 轴齿轮比为 2: 1
PR, 300	指定修正距离
SP, 5000	指定修正速度
AC, 100000	指定修正加速度
DC, 100000	指定修正减速度
BGB	开始修正

电子凸轮

电子凸轮 (ECAM) 是同样是一种使一个或几个轴的运动状态与另一个轴同步的运动控制方式。与电子齿轮不同的是, 从动轴与主动轴之间可以不是简单的比例关系。电子凸轮模式下, 从动轴与主动轴之间可以是周期性重复的任意关系。在电子凸轮模式下, 必须以主轴的编码器反馈位置作为同步的基准。

电子凸轮以轴之间的凸轮关系表为基础, 使所有控制轴实现同步。DMC-B140 控制器就可以有 1 个主动轴和 3 个从动轴。为了说明创建凸轮表的过程, 我们假设 A 轴为主动轴, B 轴为从动轴的凸轮关系。每个周期内两轴的位置关系如下图:



第 1 步. 指定主动轴

电子凸轮方式的第一步是指定主动轴, 使用如下指令:

EAp

其中: p=A、B、C、D 是选择的主动轴

对于这个例子, 主动轴为 A 轴, 指令为:

EAA

第 2 步. 规定主动轴循环周期和从动轴变化

在电子凸轮方式中, 主动轴的位置表达模式为 1 个循环周期。在本例中, A 轴位置总是表达为 0~6000 之间的数值。同样, 也对从动轴重新定义, 以便它在 0 处开始启动, 在 1500 处结束。

在循环终点, 当主动轴为 6000, 从动轴为 1500 时, 就将 A 和 B 的位置重新定义为 0, 为了指定主动轴循环周期和从动轴循环周期变化, 我们使用指令 EM。

Ema, b, c, d

其中: a、b、c、d 指定主动轴的循环周期和一个循环周期后从动轴的总变化量。

主动轴的循环周期限定在 8388607 以内, 而每个循环周期从动轴变化量限定在 2147483647。如果变化量是负数, 就取其绝对值。对于给出的例子, 主动轴的循环周期为 6000cts, 而从动轴的变化量为 1500 脉冲, C、D 两轴不参与电子凸轮模式, 指令为:

EM6000, 1500

第 3 步. 指定主动轴间隔和起始点

下一步, 我们要创建电子凸轮表, 以主动轴位置的均匀间隔规定凸轮表。最多可以有 256 个间隔。用如下指令来规定主动轴间隔大小和起始点:

EPm, n

其中: m 是间隔宽度, 单位为 cts, n 是起始点。

对于给定的例子, 我们通过规定主动轴间隔点 0, 2000, 4000, 6000 来构建凸轮表。指令为:

EP2000, 0

第 4 步. 指定从动轴位置

下一步, 我们用如下指令规定从动轴的位置:

ET[n]=a, b, c, d

其中: n 表示各点次序。

n 值以 0 开始, 最多到 256, 参数 a、b、c、d 表示相应点从动轴位置。对于此例而言, 表达如下:

ET[0]=, 0

ET[1]=, 3000

ET[2]=, 2250

ET[3]=, 1500

这样就创建 ECAM 表。

第 5 步. 使 ECAM 使能

为了使 ECAM 方式使能, 使用指令

EBn

其中: n=1 使 ECAM 方式使能, n=0 使 ECAM 方式无效。

第 6 步. 使从动轴运动啮合

要使从动轴啮合, 使用指令

EGa, b, c, d

其中: a, b, c, d 是必须使相对应的从动轴啮合处的主动轴位置。

如果任意参数值在一个循环周期范围之外, 凸轮就立即啮合。当凸轮啮合时, 自动重新定义从动轴位置, 模为 1 个循环周期。

第 7 步. 使从动轴运动脱开

要使凸轮脱开, 使用如下指令:

EQ a, b, c, d

其中: a, b, c, d 是从动轴脱开处的主动轴位置。

这就使从动轴在指定的主动轴位置点脱开。如果参数在主动轴循环周期以外, 就可瞬时停止。

第 8 步. 创建程序运行 ECAM 方式

现在假设, 从动轴以通用输入信号 1 控制啮合, 但啮合和脱开点必须在循环周期终点 A=1000, B=500 处进行, 这样就必须驱动 B 轴到该点以避免跳动。

用程序实现如下:

指令	说明
#RUN	标号
EB 1	使凸轮使能
PA, 500	起始位置
SP, 5000	B 轴速度
BG B	使 B 轴电机运动
AM	在 B 轴运动之后
AI 1	等待通用输入信号 1 输入
EG, 1000	使从动轴啮合
AI-1	等待通用输入信号 1 取消
EQ, 1000	使从动轴脱开
EN	结束

电子凸轮指令汇总

指令	说明
EA	指定 ECAM 主动轴
EB	使 ECAM 使能
EC	设置 ECAM 表的索引
EG	使 ECAM 啮合
EM	指定主动轴循环周期和从动轴一个循环周期中的变化量
EP	定义 ECAM 表的起点和间隔
EQ	使 ECAM 在指定位位置脱开
ET[n]	定义 ECAM 表
EW	电子凸轮加宽数据间隔
EY	电子凸轮循环计数
_EB	读取 ECAM 状态
_EC	读取当前 ECAM 表索引
_EG	读取各轴 ECAM 状态
_EM	读取各轴循环周期
_EP	读取 ECAM 表各项的间隔
_EQ	读取各轴 ECAM 状态

举例

下例表示具有主动轴为 C 轴，两个从动轴为 A、B 轴的 ECAM 程序：

指令	说明
#A	标号：初始化变量
PA0,0;BGAB;AMAB	A、B 轴运动到位置 0，0
EAC	C 轴作为 ECAM 主动轴
EM0,0,4000	C 轴变化量为 4000，A、B 轴为 0
EP400,0	ECAM 间隔为 400cts，以 0 为始点
ET[0]=0,0	主动轴在 0 位置时，为第 1 点
ET[1]=40,20	ECAM 表中的第 2 点
ET[2]=120,60	ECAM 表中的第 3 点
ET[3]=240,120	ECAM 表中的第 4 点
ET[4]=280,140	ECAM 表中的第 5 点
ET[5]=280,140	ECAM 表中的第 6 点
ET[6]=280,140	ECAM 表中的第 7 点
ET[7]=240,120	ECAM 表中的第 8 点
ET[8]=120,60	ECAM 表中的第 9 点
ET[9]=40,20	ECAM 表中的第 10 点
ET[10]=0,0	下一个循环周期的起始点
EB1	ECAM 方式使能
JGC=4000	设置 C 轴为 JOG 方式，速度为 4000cts / sec
EG0,0	当主动轴=0 时，使 A、B 轴啮合
BGC	开始 C 轴 JOG 运动

EN	程序结束
当需要停止电子凸轮运动时，依次输入指令：	
EQ2000, 2000	当主动轴=2000 时，使 A、B 轴脱开
STC	停止 C 轴运动
EB0	退出 ECAM 方式

轨迹方式（样条插值）

控制器也可以实现由用户规划的轨迹（样条曲线）。用户能够在 1-4 轴内定义位置-时间曲线，之后，将曲线上采样数据提供给控制器，实现这样的运动轨迹，如：抛物线、螺旋线或其它任意曲线。

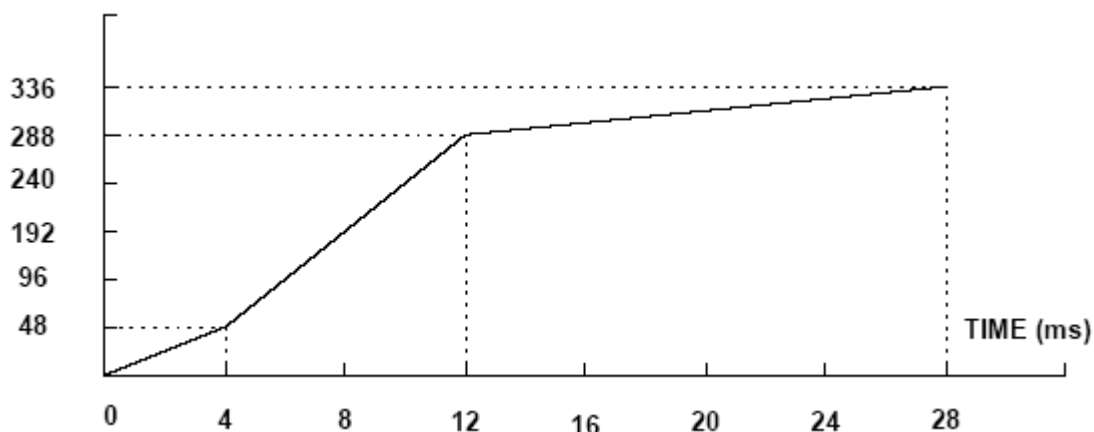
指定轨迹线段

用指令 CM 来指定轨迹方式，例如，CMAC 将 A、C 轴指定为轨迹方式。没有用于轨迹方式的其它轴可以以其它方式运动。

轨迹用位置以及时间的增量来描述。指令 CD a, b, c, d =n 来定义增量。a、b、c 与 d 为位置增量，n 为时间增量（时间间隔），为 1~8 之间的整数。时间间隔为 2^n ，单位为控制器本身的工作周期（默认状态下约为 1ms）。控制器在指定的增量之内进行直线插补，但两段增量之间没有速度的平滑处理。

指令 DTn 同样可以说明时间增量，参数 n 指定时间间隔，n 的意义与 CD 指令中的参数 n 相同。

例如，我们以图下为例，由以下各点来描述 A 轴位置。



- 点 1 T=0ms 时， A=0
- 点 2 T=4ms 时， A=48
- 点 3 T=12ms 时， A=288
- 点 4 T=28ms 时， A=336

同样的轨迹用增量表达如下：

- 增量 1 DA=48 时间=4 DT=2
- 增量 2 DA=240 时间=8 DT=3
- 增量 3 DA=48 时间=16 DT=4

当控制器收到沿这些点产生轨迹的指令时，它就在这些点之间进行直线插补。插补各点的结果为：在 1ms 处，位置值 12，在 2ms 处，位置值 24，等等，以此类推。关于上例中的程

序如下：

指令	说明
#A	标号
CMA	指定 A 轴为轨迹方式
CD48=2	指定第 1 个位置增量及时间间隔
CD240=3	指定第 2 个位置增量及时间间隔
CD48=4	指定第 3 个位置增量及时间间隔
CD0=0	设置位置、时间增量均为 0，表示退出轨迹方式
EN	程序结束

这个例子中只用到 3 组数据，CD 指令可以连续写入。控制器轨迹模式下内部的数据缓冲区只能记录 31 组数据，如数据超过 31 组，则要等到之前的数据运动完成，缓冲区中有空间时，才能继续使用 CD 指令。操作数_CM 表示当前缓冲区内可使用的空间。最大值为 32，表示缓冲区是空的，可以连续输入 31 个 CD 指令（第一条 CD 指令会立即被执行，不站用缓冲区空间，所以通常可以连续写入 32 个 CD 指令）；最小值为 1，表示缓冲区内没有空间了，至少要等新一段增量运动开始，才能执行新的 CD 指令。

如使用指令 CD 0, 0, 0, 0 =-1 表示暂停轮廓模式运动，直到有新的 DTn 指令（n 为 1~8 之间的整数）才会继续执行缓冲区中的 CD 指令

轨迹方式指令汇总

指令	说明
CM	指定参与轨迹方式的坐标轴，未参与轨迹模式的轴可以工作在其它方式
CD	指定采样点之间的间隔，位置间隔在 ± 32762 之间，时间间隔在参数在 1~8 之间。如位置间隔与时间间隔均为 0，表示退出轨迹方式，运动结束。
DT	对位置增量指定时间间隔，其中 n 是 1-8 之间的整数。0 使轨迹方式结束，如果 n 不改变，就不需要与各条 CD 指令一同指定。
_CM	缓冲区中当前可用空间

虚拟轴功能

控制器具有虚拟轴功能，名称为 N。此轴没有实际的输入、输出接口。不过，能够用如下指令对其编程：

AC、DC、JG、SP、PR、PA、BG、IT、GA、VM、VP、CR、ST、DP、RP、EA

以下是使用虚拟轴的一些例子：

1) ECAM 主动轴举例

假设：强制 A、B 轴沿着由 ECAM 表所描述的轨迹运动，并假定，ECAM 主动轴不是外部编码器但必须是可控制的变量。

通过用指令 EAN 把 N 轴定义为主动轴，并用指令设置主动轴的模（例如 EMN=4000）来实现这一要求，其次，创建凸轮表，要使强制轴运动，就简单地以 JOG 方式或指令 PR、PA 指令 N 轴。

例如：PAN=2000;BGN

会使 A、B 轴按运动周期运动到相对应的点。

2) 正弦波运动举例

假设：A 轴必须进行 10 个循环的正弦波运动，幅值为 1000 脉冲，频率为 20Hz，其它各轴不动。据此要求，我们让 A 轴和 N 轴进行圆弧运动，注意：VS 的值必须是：

$$VS=2\pi \cdot R \cdot F$$

其中 R 是半径或幅值，F 是频率，单位为 Hz。

设置 VA 和 VD 为最大值，以求最快的加、减速度。

指令	说明
VM AN	选择轴
VA 1073740800	最大加速度
VD 1073740800	最大减速度
VS 125664	VS 为 20Hz
CR 1000, -90, 3600	10 圈
VE	
BG S	开始矢量运动

运动平滑(S 曲线加减速)

控制器具有运动速度平滑功能，通常也称之为 S 曲线加/减速功能。以此来减小系统机械在加/减速过程中的冲击。

梯形速度曲线（即线性加减速）使加速度突然从 0 变到设定的最大值，不连续的加速度往往会引起振荡，产生冲击。速度曲线平滑功能产生连续加速曲线并减小机械冲击和振荡。

用 IT 指令使运动平滑：

IT 指令可以实现运动平滑功能。这些指令对加、减速函数进行滤波，以产生平滑的速度轮廓。所产生的速度轮廓具有连续加速性能，从而减小机械冲击。

用以下指令来指定平滑功能：

ITa, b, c, d 独立运动时间常数

指令 IT 用于平滑 JG、PR、PA 等非插补方式的运动。平滑参数 a、b、c、d 是 0~1 之间的数，它决定滤波强度。最大值 1 的效果为梯形速度轮廓的。平滑参数值越小意味着滤波作用越强，加/减速过程运动越平滑。

下例说明平滑效果，图形表示梯形速度轮廓及加入平滑滤波后的加速度和速度曲线。

注意：在同样的加速度、减速度数值的设置下，平滑加/减速会使加/减速的时间变长。

举例

指令	说明
PR20000	位置
ACI00000	加速度
DCI00000	减速度
SP5000	速度
IT0.5	平滑滤波器

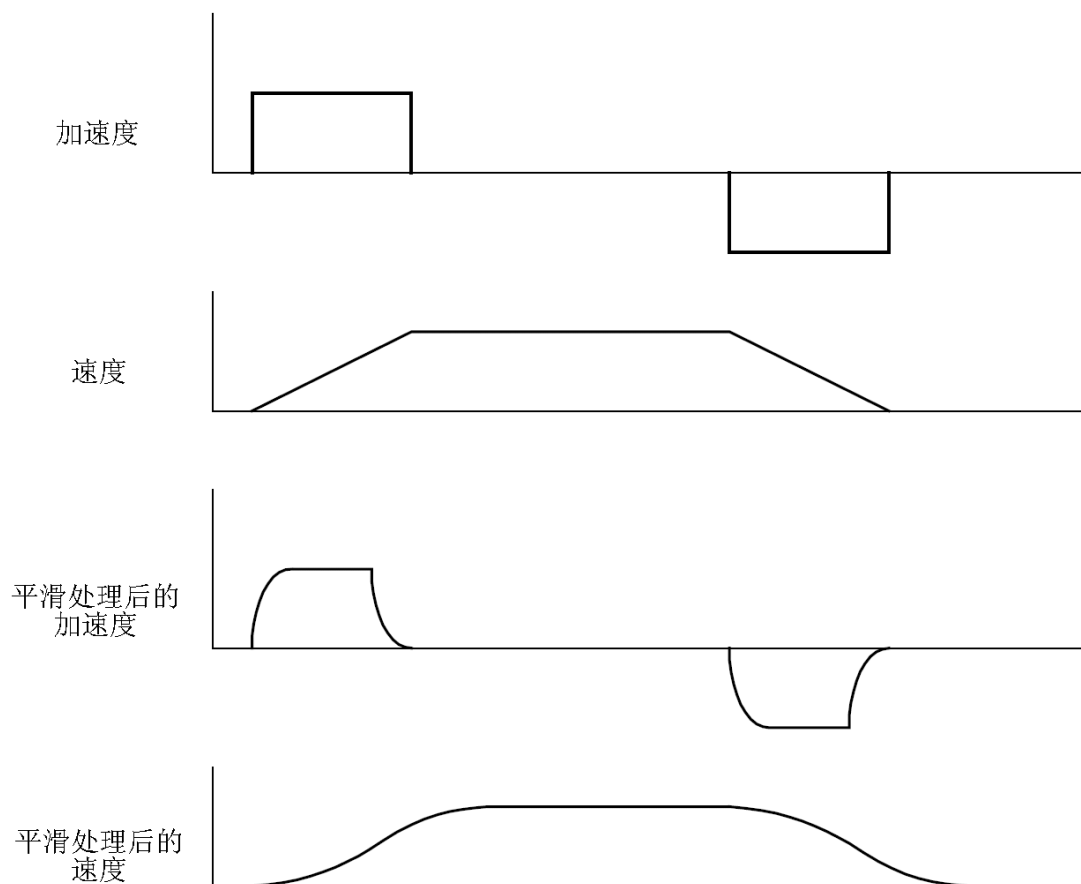
用 KS 指令使输出脉冲频率变化平缓

控制器还提供了 KS 指令对脉冲的频率进行平滑处理。KS 指令用法如下：

KS a, b, c, d

其中：a、b、c、d 是 0.5~8 之间的数，它代表平滑量（滤波深度）。数值越大，滤波的作用越强。

KS 指令与指令 IT 作用相似，可以使电机的速度变化更平缓。KS 指令与 IT 指令也有一点区别。首先，即使将各轴的参数设置为最小的 0.5，依然有滤波作用，其次，KS 的滤波作用不是在轨迹规划的过程中起作用，而是在轨迹规划之后，对脉冲输出起作用。这样早成的结果是，运动过程中的脉冲输出略微滞后于轨迹规划。KS 参数值越大，这种滞后越大。在匀速运动过程中，滞后大约是 KS 参数值*3 个控制周期。



原点返回功能

对于一个运动系统来说，原点返回功能是非常重要的。一台设备每次开始工作之前，都要确定坐标原点的位置。有的设备允许以任何位置作为原点，但大多数设备要求以一个固定的位置作为原点。

如何在运动系统的整个行程中确认一个原点的位置呢？比较简单的办法，是在机械上的特定位置安装一个传感器作为原点信号开关，当运动机构经过这个位置时，开关会输出不同

的信号。这样，在信号变化瞬间机构所在的位置就是系统坐标的基准。

由于原点开关不同的工作方式以及应用条件的限制，这样得到的原点位置每次不会完全一致，可能在几个微米到几百个微米的范围内变化。对于重复性要求比较高的系统，原点开关信号本身的重复性不可接受的。对于这样的系统，通常使用增量位置反馈编码器的零位信号来确认原点的位置。这样得到的原点位置的重复性，一般可以达到反馈系统一个分辨率的程度。这里所说的编码器，不单纯指旋转编码器，而是泛指一切实现位置反馈的系统部件，比如光栅尺、磁栅尺等等。零位信号（ZERO）有时也被称为索引信号或标志信号（INDEX）。

采用编码器的零位信号作为原点的标志，也可能存在问题。这个信号在系统的整个运动范围内很可能不止出现一次。若将错误的信号作为原点标志，带来的后果会是非常严重的。所以，比较经典的原点返回动作，通常是首先找到感应开关信号变化的位置，而后向固定的方向移动指定的距离，再继续移动到编码器零位信号出现的位置，并以此作为运动系统的坐标原点基础。

之所以要移动一段距离，是为了避免出现一种特殊的情况：编码器的零位信号恰好出现在感应开关位置不确定的范围内。如果真的出现这种情况，用户可以通过改变移动的距离，避开不确定的区域，以另外一个零位信号作为运动系统的坐标基准。

GALIL 控制器与原点用于原点返回的指令有三个，FE、HM、FI。FE 与 HM 用于确认原点开关的位置，FI 用于确认零位信号的位置。DMC-B140-M 控制器为每轴提供了一个原点开关的电气接口，也为每轴提供了编码器零位信号的接口。

HM 指令用于确认原点开关的位置，并将其作为系统的原点。HM 和 BG 指令的动作过程如下：

1. 开始运动，电机加速到工作速度，运动方向由原点输入的状态来决定。原点输入信号导通使电机向正方向运动，原点输入信号截止或悬空使电机向反方向开始运动。（用 CN 指令可以改变这一关系）。
2. 检测到原点开关信号改变状态，电机减速停止。
3. 电机停止后，以 256 的速度慢慢返回。
4. 检测到原点开关信号恢复原状态，电机立即停止并以当前位置为原点（将位置值设置为 0）。

FI 指令的作用是运动到编码器零位信号处，并以其位置作为原点。它需要与 JG 指令配合使用，由 JG 指令确认运动的方向与速度。控制器能接收的最短的零位信号是 80 纳秒，如果零位信号持续的时间小于 80 纳秒，控制器有可能接收不到。

FE 指令用于确认原点开关的位置。执行 FE 和 BG 指令时，电机就会加速到 SP 指令设置的速度，移动至检测到原点开关输入信号状态变化后，电机就会减速停止。

HM 可以较精确的确认原点开关的位置，当以原点开关作为系统坐标基准时，它是最方便的指令。FE 用于大致确认原点开关的位置，主要用来与 FI 配合使用。当然，无论使用以上三个指令中的哪一个，如果在运动中，运动方向对应的极限信号有效，都会立即减速停止，并不能确认原点。所以，在动作完成后，可能需要确认动作结束是由于正常返回到了原点位置，还是由于极限信号或其它原因。

在原点返回功能中，还有一个很重要的操作数_HMn，它表示当前原点开关信号的输入状态。

举例

指令	说明
#HOME	标号
ACI000000	加速度
DCI000000	减速度
SP5000	回原点速度
HM A	A 轴原点返回
BG A	开始运动
AM A	运动完成后
MG "A IS AT HOME"	发送信息
EN	结束
#ZERO	标号
ACB=2000000	加速度
JGB=8000	向正方向，高速运动
FIB	寻找零位信号
BGB	开始
AMB	运动完成后
IPB=100	速度
JGB=-256	低速倒退，精确寻找零位信号
FIB	搜寻原点开关边缘指令
BGB	开始运动
AMB	运动完成后
MG "B IS AT HOME"	
EN	
#H2Z	标号
ACC=2000000	加速度
DCC=2000000	减速度
SPC=8000	速度
FEC	搜寻原点开关边缘
BGC	开始运动
AMC	运动完成后
JGC=256	
FIC	
BGC	
AMC	
MG "C IS AT HOME"	发送信息
EN	结束

原点返回指令汇总

指令	说明
FE	搜寻原点开关信号变化边缘
FI	搜寻索引信号，并将其作为原点

HM	以原点开关信号切换的位置为基准，返回原点
SC	查询停止代码
TS	查询各开关状态
_HMx	“x”轴原点开关当前状态
_SCx	“x”轴停止代码
_TSx	“x”轴相关各个开关状态

位置锁存（高速位置捕捉）功能

对于印刷套色等需要对标的应用来说，往往需要精确的知道外部信号输入时运动机构的精确位置。DMC-B140-M 控制器为此提供了位置锁存功能。当外部信号变化时，精确锁存到这一瞬间运动机构所在的位置，用户可以在之后查询这个位置数值。每个轴都有一个特定的输入信号用于触发锁存动作。对于 A、B、C、D 四轴而言，分别是通用输入 1、2、3、4。可以用 CN 指令选择输入信号导通或截止的瞬间位置进行锁存，也可以选择锁存编码器反馈的位置或输出脉冲的位置。对于编码器反馈位置，也可以选择用零位信号来锁存。

用指令 AL 指令用来设置告诉位置捕捉功能。AL 指令有三中方式

AL ABCD 由通用输入信号触发相应轴对编码器反馈位置的锁存

AL SASBSCSD 由通用输入信号触发相应轴对输出脉冲位置的锁存

AL SASBSCSD 由零位信号触发相应轴对编码器反馈位置的锁存

操作数_ALn 代表锁存动作是否完成，如果已经准备后，但还未进行过位置锁存，操作数为 1，如已经完成了位置捕捉，操作数为 0。

RL 是一个查询指令，用于查询锁存到的位置值。例如：

RL A 或 RL CD

操作数_RLn 的值同样为锁存到的位置值。

下面通过一个例子，用 DMC-B140-M 控制器的通用输出信号来锁存输出脉冲的位置，演示 AL 与 RL 的使用方式。

首先，在 DMC-B140-M 控制器 IO 接口中的引脚做如下连接：7、8 脚短接；23、25 脚短接；4、36 脚短接。控制器上电后，依次输入以下指令：

JG, 5000	JOG B 轴
BGB	
ALSB	B 轴锁存使能
MG _ALB	检查位置锁存是否完成，应反馈 1
SB2	
CB2	
MG _ALB	检查位置锁存是否完成，应反馈 0
RLB	显示锁存结果
MG _RLB	再次显示锁存结果

高速位置捕捉指令汇总

指令	说明
AL	设置位置锁存功能
RL	查询锁存的位置值
_ALx	位置锁存功能状态

_RLx	锁存的位置值
------	--------

第七章 编程

概述

GALIL 运动控制器本身具有处理器及 Flash、RAM 存储器。控制器独立可以完成指令解释、用户程序执行、运动规划、I/O 事件处理、高速插补运算、通信管理等。我们可以把程序下载、保存到 DMC-B140-M 的存储器中，脱离主机运行。不过，如果需要，在执行程序时，仍然能够随时给控制器发送指令。

除基本的运动之外，DMC-B140-M 还提供了一些功能，使用户程序可以进行数学、逻辑运算，对 I/O 进行控制，控制通讯，也可以使用变量、数组这样的数据结构，实现条件进行判断、分支、跳转（循环）等程序结构。

在第五章中，已经对程序的基本组成进行了介绍。程序由语句行组成。语句行可以包含标号与指令等内容。用于直接实现运动控制的指令，大部分已经在前一章中介绍过了，本章主要介绍一些数据处理、流程控制方面的指令与函数。将这些功能与控制指令结合使用，才能充分发挥出 DMC-B140-M 独立控制器的真正能力。

程序格式

程序由语句行组成。语句行可以包含标号与指令以及数学运算赋值等内容。用分号把一行中的多条指令分开。

DMC-B140-M 的程序存储空间中，每行只能存储 40 个字符。对于一些比较复杂的判断或运算，长度要超过 40 个字符。对于这种情况，DMC-B140-M 控制器提供了用于连接语句行的专用符号——`（ASCII 代码 96，不是单引号）。这个符号出现在语句行的末尾，表明本行逻辑上并没有结束，下一行的内容与本行是连续的。例如，下面的内容虽然是两行，但是控制器会正确的理解并执行其中的指令：

```
PA200000000, 2000000000, 2000000000`  
2000000000
```

如果一个语句行中间出现这个符号，那么该符号后面的字符被忽略。

标号与注释

建议所有程序以标号开始，以 EN 语句结束。标号以“#”开始，后面最多可跟随 7 个字符的标号名称。标号名称第一个字符必须是字母，其后允许使用数字，但不允许有空格。

有效标号举例：

```
#BEGIN  
#SQUARE  
#X1  
#BEGIN1
```

无效标号举例：

```
#1Square  
#123
```


对于用户的程序而言，完全不使用标号也可以。在下面的介绍中，可以看到，标号的主要作用体现在流程控制方面，而这个作用通常也可以用行号代替。不过，对于不断编辑、调整的程序来说，使用标号会比使用行号方便得多。

程序中不能使用相同的标号，出现在这种情况，控制器会提示错误。DMC-B140-M 控制器允许用户的程序中使用最多 62 个不同的标号。

特殊标号

DMC-B140-M 控制器中有一些特殊标号，这些标号对控制器而言代表着特定的含义。这些标号后的语句行在特定情况下回自动执行。如果用户在自己的程序中错误的使用了这些标号，可能会导致程序的执行过程与所期望的不同。（参见“自动启动子程序”一节）

以下是这些标号的列表：

```
#AUTO
#AUTOERR
#CMDERR
#COMINT
#ININT
#LIMSWI
#TCPERR
```

注释程序

注释程序有两种方法，第一种方法使用 NO 指令，可以在 GALIL 程序内直接使用；第二种方法是使用 REM 语句，这需要使用 GALIL 软件来进行。

NO 指令

DMC 控制器提供 NO 指令用于对程序注释。举例如下：

指令	说明
#PATH	标号
NO2-D CIRCULAR PATH	注释——无操作
VM AB	矢量方式
NO VECTOR MOTION ON A AND B	注释——无操作
VSI0000	矢量速度
NO VECTOR SPEED IS 10000	注释——无操作
VP-4000, 0	矢量位置
NO BOTTOM LINE	注释——无操作
CRI500, 270, -180	圆弧运动
NO HALF CIRCLE MOTION	注释——无操作
VP0, 3000	矢量位置
NO TOP LINE	注释——无操作
CRI500, 90, -180	圆弧
NO HALF CIRCLE MOTION	注释——无操作
VE	矢量结束
NO END VECTOR SEQUENCE	注释——无操作
BGS	开始运动
NO BEGIN SEQUENCE MOTION	注释——无操作

EN

结束程序

NO END OF PROGRAM

注释——无操作

注意：NO 指令是实际控制器指令，因此，NO 指令会占用控制器的处理时间。

提示：有些用户“NOTE”对程序加注，对于控制器而言，实际上“NO”之后“TE 两个字符也作为注释来处理。

REM 指令

如果您使用 GalilTools 软件连编辑程序，那么也用 REM 语句来注释程序。‘REM’ 语句以字‘REM’ 开始，在同一行中可以跟随任何注释。当下载程序到控制器时，GalilTools 会去掉这些语句。例如：

```
#PATH
REM2-DCIRCULAR PATH
VMAB
REM VECTOR MOTIONON A AND B
VS 10000
REM VECTOR SPEED IS 10000
VP-4000, 0
REM BOTTOM LINE
CR 1500, 270, -180
REM HALF CIRCLE MOTION
VP0, 3000
REM TOP LINE
CRI500, 90, 180
REM HALF CIRCLE MOTION
VE
REM END VECTOR SEQUENCE
BGS
REM BEGIN SEQUENCE MOTION
EN
REM END OF PROGRAM
```

当下载此程序到控制器时，如果用 LS 指令或上载的方式查看程序，会发现这些 REM 语句都不存在了。

```
#PATH
VMAB
VS 10000
VP-4000, 0
CR 1500, 270, -180
VP0, 3000
CRI500, 90, 180
VE
BGS
EN
```

运行程序——多任务

DMC-B140-M 控制器可以为用户提供 4 个线程运行程序，从 0~3 进行编号。这些线程可以同时用于执行程序（控制器内部分时处理），实现多任务功能。这一功能对于执行相对独立的操作，如 PLC 功能非常有用。

在 4 个线程中，0 号是主线程。主线程与其它线程在以下方面有所不同：

- (1) 只有主线程（0 线程）可以使用指令 IN
- (2) 限位开关处理、位置超差、指令出错等子程序均自动在 0 线程执行。

要开始运行程序，使用如下指令：

XQ#A, m 或者 XQn, m 其中：#A 为标号；n 为行号；m 为线程号

程序将使用指定的线程，从指定的标号或行号开始运行。如果不指定行号或标号，则默认从首行（行号为 0）开始；如不指定线程号，则默认利用 0 线程。如果指定（或默认）的线程已经在运行程序了，则控制器会提示这个 XQ 指令出错。

要中止任意线程中程序的运行，用指令：

HX n 其中 n 是线程号

如果不指定线程号，则中止所有线程。

需要注意的是，HX 指令只是中止程序的运行，并不会改变之前程序运行的结果。例如，程序中执行了 JG10000; BG 这样的指令后，HX 并不能使 A 轴的电机停止。

XQ 与 HX 也可以作为指令写入程序中。

下例是利用多任务同时在 4 个通用输出点产生不同宽度脉冲输出的程序。

指令	说明
#TASK0	标号
XQ#TASK1, 1	在 1 线程内执行以标号#TASK1 开始的语句行
XQ#TASK2, 2	在 2 线程内执行以标号#TASK2 开始的语句行
XQ#TASK3, 3	在 3 线程内执行以标号#TASK3 开始的语句行
SB1	使通用输出 1 导通
WT500	等待 0.5 秒
CB1	使通用输出 1 截止
EN	执行结束
#TASK1	标号
SB2	使通用输出 2 导通
WT1000	等待 1 秒
CB2	使通用输出 2 截止
EN	执行结束
#TASK2	标号
SB3	使通用输出 3 导通
WT700	等待 0.7 秒
CB3	使通用输出 3 截止
EN	执行结束
#TASK3	标号
SB4	使通用输出 4 导通
WT800	等待 0.8 秒
CB4	使通用输出 4 截止
EN	执行结束

用 XQ#TASK, 0 或简单的 XQ 指令执行上述程序。

调试程序

DMC-B140-M 提供了专门用于调试应用程序的指令和操作数。这些指令有用于监测程序执行的指令，也有确认控制器状态和控制器程序、数组和变量空间大小的查询指令。操作数也包含有助于调试程序的重要状态信息。

跟踪指令

跟踪指令会在程序执行之前，使控制器将程序中的每一行立即发送到主计算机。用指令 TR1 使跟踪功能使能，TR0 使跟踪功能关断。

从控制器送出的数据存储在输出 FIFO 缓冲区中。输出 FIFO 缓冲区能存储 512 个字符信息。在正常运行中，控制器把输出内容放入 FIFO 缓冲区，主计算机侧的软件监测缓冲区并读取所需信息。当跟踪方式使能时，控制器就以非常高的速率把这些信息发送到 FIFO 缓冲区。一般来说，由于软件读取信息较慢，因此 FIFO 就可能会放满。当 FIFO 放满时，如不清除，就会使程序执行速度减慢。如果用户想避免这种不必要的延迟，就发送指令 CW, 1，此指令使控制器把那些不能放入 FIFO 的数据扔掉。在此情况下，控制器就不会延迟程序执行。

错误处理指令

当有程序错误时，程序在错误发生点中止。要显示程序执行的最后一行的行号就用指令 MG_ED。

用户用指令 TCI 能得到错误的类型，此指令报告错误代码号和描述出错内容的文本信息，指令 TC0 或 TC 返回不带文本信息的错误代码。详见《指令手册》中有关 TC 指令的说明。

停止代码指令

用停止代码指令 SC 能确认各轴的运动状态，当某轴运动莫名其妙停止时，此指令很有用。SC 指令会返回代表运动状态的代码号。详见《指令手册》。

存储空间查询指令

为了使用户了解程序存储空间，数组存储空间或变量存储空间的状态，DMC-B140-M 有几个很有用的指令。指令 DM? 返回当前可用的数组元素数；指令 DA? 返回当前能被定义的数组数。例如，控制器最多可以存储 6 个数组，800 个数组元素。如果已定义了一个 100 个元素的数组，则 DM? 指令返回值为 700，DA? 指令返回值为 5。

列出所有已经定义的变量，使用查询指令 LV；列出已经定义的数组，使用查询指令 LA；列出程序区的内容，使用查询指令 LS（列表）；只列出应用程序中使用的标号，使用查询指令 LL（标号列表）。

（关于数组与变量的应用细节，在本章后面介绍）

操作数

一般来说，所有操作数在调试应用程序中均可提供有用的相关信息。下面列出的这些操作数对程序调试特别有用。为了显示操作数的值，可以使用 **MG** 指令。例如，由于操作数 **_ED** 读取程序执行的最后一行，那么，指令 **MG_ED** 就会显示此行号。

_ED	读取程序执行的最后一行。用于确定程序停止的地方。
_DL	读取可用的标号数
_UL	读取可用的变量数
_DA	读取可用的数组数
_DM	读取可用的数组元素数
_AB	读取急停输入状态
_FLa	读取 ‘a’ 轴正向限位开关的状态
_RLa	读取 ‘a’ 轴负向限位开关的状态

举例

下例有一个错误，在 A 轴处于运动中时，试图指定新的相对位移量。当执行此程序时，控制器停在 003 行，然后，用户就可以用指令 **TCl** 对控制器发问查询，控制器就会以相对的说明给予回应：

指令	说明
#A	标号
PR1000	相对位置 1000
BGA	开始
PR5000	相对位移 5000
EN	END

执行此程序，就会看到信息：

?003

提示第 3 行有错误。此时要检查具体是什么错误，输入指令

TCl

显示信息

?7Command not valid while running

程序流程指令

DMC-B140-M 提供了控制程序流程的指令，**DMC-B140-M** 通常按顺序执行程序中的指令。使用事件触发、条件启动、条件跳转、逻辑判断语句可以改变执行的过程。

事件触发

事件触发是指控制器暂停执行程序，直到指定的事件出现，才继续程序的运行。这样的事件有如下几种：某运动完成、定时到达、某输入逻辑改变。

例如，能够用运动完成触发器把程序中的两个运动过程分开；直到第一个运动过程完成才会执行用于第二个运动过程的指令。

事件触发指令

指令	说明
AM	直到所指定轴或运动按序完成程序才继续执行。不带参数的 AM 适用于所有轴的运动完成，此指令对分开程序中的运动过程很有用。
AD	使程序执行暂停，直到位置指令到达所指定的相对于运动始点的距离。一次只能指定一个轴
AR	使程序执行暂停，直到由上一个 AR 或 AD 指令所指定的距离过后，才执行后续指令。一次只能指定一个轴。
AP	使程序执行暂停，到达指定绝对位置之后，才继续执行。每次只能指定一个轴。
MF	使程序执行暂停，直到正向运动到达指定绝对位置后才继续执行。一次只能指定一个轴。如果位置已经越过指定位置点，MF 会立即释放。此功能对齿轮轴或辅助编码器输入起作用。
MR	使程序执行暂停，直到反向运动到达指定绝对位置之后才继续执行。一次指定一个轴。如果位置已越过指定位置点，MR 会立即释放。此功能对齿轮轴或辅助编码器输入起作用。
MC	使程序执行暂停，直到脉冲输出完成才继续执行。
AI	使程序执行暂停，直到所指定的输入处于所规定的逻辑电平，n 指定输入点。正为外部截止或悬空状态，负为导通状态。
AS	使程序执行暂停，直到所指定轴达到指定速度
AT	使程序执行暂停，直到从计时起点后 n 毫秒才继续执行。AT0 设置计时起点，ATn 从计时起点开始等待 n 毫秒。AT-n 从参计时起点开始等待 n 毫秒，并在时间到达之后设置新的计时起点。
AV	使程序执行暂停，直到沿联动轨迹指定的距离出现
WT	使程序执行暂停，直到所指定的等待时间完成。

举例：

1) 多重运动过程

用 AM 条件启动指令把两个 PR 运动分开，如果不使用 AM 指令控制器在执行第二个 PR 指令时会提示出错。

指令	说明
#TWO MOVE	标号
PR2000	相对位置值 2000
BGA	开始运动
AMA	等待运动完
PR4000	下一位置运动
BGA	开始第二个运动
EN	程序结束

2) 在距离之后设置输出

在离开运动始点 1000cts 距离之后设置通用输出 1 导通。条件启动的精度是控制器工作周期乘以速度的两倍。

指令	说明
#SETBIT	标号

SPI0000	速度为 10000
PA20000	绝对位置
BGA	开始运动
ADI000	等待 1000cts 到达
SBI	设置通用输出 1 导通
EN	程序结束

3) 重复性位置触发

在运动期间，每 10000cts 设置输出位，用 AR 条件启动。

指令	说明
#TRIP	标号
JG50000	指定 JOG 速度
BGA;n=0	开始运动
#REPEAT	重复循环#REPEAT
ARI0000	等待 10000cts 到达
TPA	报告位置
SB1	设置输出 1
WT50	等待 50 毫秒
CB1	输出 1 截止
n=n+1	循环计数+1
JP#REPEAT, n<5	重复循环 5 次（JP 指令会在下一节介绍）
STA	停止
EN	程序结束

（此例中使用了跳转指令构成循环，跳转指令的应用将在后文中详细介绍。）

4) 按外部输入开始运动

此例等待通用输入 1 导通，然后开始运动

指令	说明
#INPUT	程序标号
AI-1	等待输入点 1 导通
PR10000	相对位置 10000
BGA	开始运动
EN	程序结束

5) 当速度到达时，设置输出点

指令	说明
#ATSPEED	程序标号
JG50000	指定 JOG 速度
ACI0000	加速度
BGA	开始运动
ASA	等待速度到达 50000
SB1	通用输出 1 导通
EN	程序结束

6) 改变矢量轨迹速度

下列程序使进给率或矢量速度在指定的矢量运动距离点发生改变。矢量距离从运动始点

或上一个 AV 指令开始计算：

指令	说明
#VECTOR	标号
VMAB;VS5000	联动轨迹
VPI0000, 20000	矢量位置
VP20000, 30000	矢量位置
VE	矢量结束
BGS	开始运动
AV5000	在矢量距离 5000 到达之后
VSI000	减速
EN	结束

7) 带有等待的多重运动

本例通过在执行新的运动之前等待各个运动完成进行多重相对位移运动：

指令	说明
#MOVES	标号
PR12000	距离
SP20000	速度
ACI00000	加速度
BGA	开始运动
ADI0000	等待 10000cts 距离到达
SP5000	新速度
AMA	等待运动完成
WT200	暂停 200ms
PR-10000	新位置
SP30000	新速度
ACI50000	新加速度
BGA	开始运动
EN	结束

8) 用 AT 定义输出波形

下列程序在通用输出 1 产生 10ms 导通，40ms 截止的脉冲列，每 50ms 重复一次。

指令	说明
#OUTPUT	程序标号
AT0	初始化时间参考点
#LOOP	循环
SB1	通用输出 1 导通
ATI0	从参考点计时 10ms 之后
CB1	通用输出 1 截止
AT-40	从参考点计时等待 40ms 并重新设置参考点
JP#LOOP	循环
EN	结束

跳转 JP

控制器的程序除顺序执行外，也具有跳转功能，对应指令 **JP**。使程序根据所指定的条件跳转到不同的分支中去执行。与事件触发不同，跳转指令不暂停程序。条件跳转功能对于实时检测事件并进行相应处理很有用。例如，DMC-B140-M 能以输入点状态为条件在两种运动轮廓之间做出选择

JP 指令格式为

JP 目标，逻辑条件

目标是当指定条件满足时程序将要跳转到的行号或标号。注意：程序首行行号为 0。

逻辑条件

逻辑条件可以是一个比较式，也可以是任何有效的 Galil 操作数、变量、数组元素、常数值、函数、算术表达式等。对于任何非 0 的结果，均意味着条件满足。如果不给出逻辑条件，即为无条件跳转。

比较式可使用如下比较符号

<	小于
>	大于
=	等于
<=	小于等于
>=	大于等于
<>	不等于

以下是一些比较式的例子。

数字	V1=6
数字表达式	V1=V7*6 @ABS[V1]>10
数组元素	V1<Count[2]
变量	V1<V2
操作数	_TPA=0 _TVA>500
I / O	V1>@AN[2] @AN[1]=0

多重条件

DMC-B140-M 可以在一个跳转语句中检查多个条件。使用“&”（与）和“|”（或）就能把多个条件语相迭加，任意两个条件之间的“&”操作数要求两个语句都必须为真。任意两个条件之间的“|”操作数要求只要任何一个语句为真。

注意：必须把各个条件放在圆括弧内，以便控制器进行正确判断。关于数字表达式及位操作算子“&”和“|”的详细介绍，请参见“数学表达式”一节。

例如，使用以 V1、V2、V3、V4 命名的变量：

JP#TEST, (V1<V2)&(V3<V4)

在此例中，若 V1 小于 V2 且 V3 小于 V4，程序就跳转到标号为#TEST。为了进一步说明，我们再看看下例：

JP#TEST, ((V1<V2)&(V3<V4)) | (V5<V6)

如果满足两个条件之一：V1 小于 V2 且 V3 小于 V4; V5<V6，程序就会跳转到标号#TEST。

举例

1) 如果满足 JP 指令的条件, 控制器就跳转到所指定的标号或行号, 并从此点开始继续执行。如果条件不满足, 控制器就按序执行下一条指令。

指令	说明
JP#LOOP, count<10	如果变量 count 小于 10, 跳转到#LOOP
JS#MOVE2, @IN[1]=1	如果通用输入 1 截止, 跳转到子程序#MOVE2。 在执行完#MOVE2 子程序之后, 程序返回到调用子程序处的主程序位置。
JP#BLUE, @ABS[V2]>2	如果变量 V2 的绝对值大于 2, 跳转到#BLUE
JP#C, V1*V7<=V8*V2	如果 V1 值乘 V7 小于或等于 V8 乘 V2 的值, 跳转到#C
JP#A	无条件跳转到#A

2) 使 A 轴电机运动到绝对位置 1000, 并返回到 0, 重复 10 次, 在每两次运动之间暂停 100ms。(由条件跳转构成循环)

指令	说明
#BEGIN	程序标号
Count=10	循环计数器置初值
#LOOP	循环程序标号
PA1000	绝对位置 1000
BGA	开始运动
AMA	等待运动完成
WTl00	暂停 100ms
PA0	返回绝对位置 0 点
BGA	开始运动
AMA	等待运动完成
WTl00	等待 100ms
Count=Count-1	循环计数器减 1
JP#LOOP, Count>0	通过循环测试 10 次
EN	程序结束

条件判断 IF、ELSE、ENDIF

DMC-B140-M 提供了对使用 IF、ELSE、ENDIF 指令的条件语句的结构化方法。

使用 IF、ENDIF 指令

IF 条件语句由 IF 和 ENDIF 指令组合来构成, IF 指令依据一个或更多的条件来判断。如果条件判断为真, 指令解释器就会继续执行跟随在 IF 指令之后的指令; 如果条件语句判断为假, 控制器就会忽略掉在 IF 指令之后的指令, 去执行与之相配合的 ENDIF 指令或者去执行在程序中出现的 ELSE 指令后的指令 (见以下相关 ELSE 指令介绍)。

注意: 对于已执行的每个 IF 指令, 总是必须执行一次 ENDIF 指令。建议用户不要在 IF 条件语句里包含跳转指令。在此情况下, 会由于执行了 IF 而未执行 ENDIF 产生错误。

使用 ELSE 指令

ELSE 指令是 IF 条件语句的选择部分, 只有当 IF 指令的条件判断为假时才执行 ELSE 指令后的指令。ELSE 指令必须出现在 IF 指令之后, 且没有条件表达式。如果 IF 指令的条件判断为假, 控制器就会跳过后续指令去执行 ELSE 和 ENDIF 之间的指令。如果用于 IF 指令的参

数判断为真，控制器就会执行 IF 和 ELSE 指令之间的指令。

嵌套 IF 条件语句

DMC-B140 允许 IF 条件语句包含在其它 IF 语句之内，称之为“嵌套”。DMC-B140 允许 255 个 IF 条件语句进行嵌套。这是一项功能非常实用的技术，使用户能对许多种不同的分支情况进行指定。

指令格式 IF ELSE ENDIF

格式	说明
IF 条件表达式	如果条件为真，就执行 IF 指令后（直到 ELSE 指令）的指令，否则，继续在 ENDIF 指令或可选的 ELSE 指令处执行。
ELSE	可选择指令，当 IF 指令的参数判断为假时，转而执行此指令，只能与 IF 指令配合使用。
ENDIF	用来使 IF 条件语句结束的指令。对于每条 IF 指令，程序必须有一个 ENDIF 指令与之对应。

指令	说明
#TEST	开始主程序“TEST”
II, 3	由通用输入 1、2 触发中断
MG”WAITING FOR INPUT1, INPUT2”	输出信息
#LOOP	
JP#LOOP	循环
EN	主程序结束
#ININT	输入中断子程序
IF(@IN[1]=0)	判断通用输入 1 状态
IF(@IN[2]=0)	若第 1 个 IF 条件为真，执行第 2 个 IF
MG”INPUT1 AND INPUT2 ARE ACTIVE”	若第 2 个 IF 条件为真，执行信息发送
ELSE	用于第 2 个语句的 ELSE 指令
MG”ONLY INPUT1 IS ACTIVE”	若第 2 个 IF 条件为假，执行信息发送
ENDIF	第 2 个条件语句结束
ELSE	用于第 1 个语句的 ELSE 指令
MG”ONLY INPUT2 IS ACTIVE”	若第 1 个 IF 条件为假，执行信息发送
ENDIF	第 1 个条件语句结束
#WAIT	用于循环的标号
JP#WAIT, (@IN[1]=0) (@IN[2]=0)	循环直到输入点 1 和 2 均无效
RI0	中断返回

子程序

所谓子程序就是一组以标号开始，且以 EN 指令结束的指令。子程序从主程序中用跳转子程序指令 JS，后跟标号或行号和条件语句进行调用。子程序允许递归，调用可以嵌套 16 层，在执行完子程序之后，程序返回到调用子程序的地方。但子程序堆栈被变换的情况除外。

例如画出边长为 500cts 正方形的子程序举例如下，正方形以矢量位置 1000，1000 处开始绘制。

指令	说明
#M	主程序标号
CB1	输出点 1 截止（抬笔）

VMAB	
VP1000, 1000;LE;BGS	定义矢量位置：移动笔
AMS	等待运动完成
SB1	输出点 1 导通（下笔）
JS#SQUARE;CB1	跳到 SQUARE 子程序
EN	结束子程序
#SQUARE	SQUARE 子程序
V1=500;JS#L	定义边长
V1=-V1;JS#L	切换方向
EN	结束子程序
#L;PRV1, V1;BGA	定义 A、B 轴位置，开始 A 轴运动
AMA;BGB;AMB	在 A 轴运动完成后，开始 B 轴运动
EN	结束子程序

堆栈变换

可以用 ZS 指令使子程序堆栈进行变换。每次 JS 指令、执行一次中断或自动子程序（如 #LIMSWI），子程序堆栈指针就加 1。通常是 EN 指令结束子程序时使堆栈指针减 1。偶尔，希望不返回到子程序或中断被调用的程序行。ZS1 指令使堆栈指针减 1，这样就能使程序定序器继续执行下一行。ZS0 指令把堆栈复位到它的初始值。

在一些特殊情况下，例如意外情况的处理子程序中，如处理后希望程序重新开始执行，而不是返回发生意外情况的位置。要满足这种要求，子程序中使用 ZS0 指令，并用 JP 指令跳转到程序的开头。

自动启动子程序

DMC 控制器有一个特殊标号用于自启动子程序执行。已经存入控制器非易失存储器中以标号 #AUTO 开始的程序在上电或复位时能够自动被运行。此程序必须用 BP 指令存入非易失性存储器中。

用于监测条件的自动子程序

经常需要不用主机或 DMC-B140-M 程序介入而连续监测某些条件，DMC-B140-M 能够在后台监测几个重要条件。这些条件有：检查限位开关、预定义的输入、指令错误等。在应用程序中插入特殊、预先定义的标号使自动监测功能生效。

预先定义的自动子程序标号有

子程序标号	说明
#LIMSWI	任意轴限位开关有效
#ININT	由 II 所规定的输入有效
#CMDERR	指令错误
#COMINT	通讯中断
#TCPERR	TCP/IP 通讯错误
#AUTOERR	执行 #AUTO 程序错误

例如，当任意一个轴运动中触发了位置极限信号，就会自动执行 #LIMSWI 子程序。用户

可以在#LIMSWI 子程序中检查出哪个极限信号被触发并采取恰当行动。在另一个例子中，用#ININT 标号来指向输入中断子程序。当指定的输入出现时，就会执行该程序。

注意，DMC-B140-M 必须正在执行某段程序，以上子程序才会在条件出现时自动运行。正在执行的可以是一个非常简单的程序，没有任何操作只是空循环，例如#LOOP; JP#LOOP; EN。

举例

1) 限位开关

限位开关一出现，此程序就打印信息。

指令	说明
#LOOP	虚拟程序
JP#LOOP: EN	跳转到循环程序
#LIMSWI	限位开关标号
MG"LIMITOCCURRED"	打印信息
RE	返回主程序

执行此程序，然后输入指令

JG5000

BGA

现在，当 A 轴正向限位开关有效时，就会执行#LIMSWI。

几点注意事项：

- 1) 用 RE 指令从#LIMSWI 子程序返回
- 2) 当子程序结束后，如果限位开关保持有效状态，就会再次执行#LIMSWI 子程序
- 3) 只有电机按照指令运动时，才有可能是极限信号有效，从而执行#LIMSWI 子程序。

2) 输入中断

指令	说明
#A	标号
III	由通用输入 1 产生输入中断
JG30000,,,60000	Jog 运动
BGAD	开始运动
#LOOP;JP#LOOP;EN	循环，等待中断
#ININT	输入中断
ST AD;AM	停止运动
#TEST;JP#TEST, @IN[1]=0	若通用输入 1 仍为导通状态，循环测试
JG3000,,,6000	Jog 运动
BGAD	开始运动
RI0	从中断程序返回到主程序，并不再允许触发中断

3) 指令错误

指令	说明
#BEGIN	开始主程序
IN"ENTERSPEED", SPEED	提示输入速度
JG SPEED;BGA	开始运动
JP#BEGIN	重复循环
EN	结束主程序

#CMDERR	指令错误子程序
JP#DONE, _ED<>2	检查第 2 行是否出错
ZS1	如是其它错误, 则调整堆栈
JP#BEGIN	返回到主程序
#DONE	第 2 行出错, 结束程序
ZS0	清除堆栈
EN	结束程序

上述程序提示操作人员输入 JOG 速度值, 如果操作人员输入的数值超出范围, 大于或 3M, 通过执行#CMDERR 子程序, 提示操作人员输入一个新速度值。

在多任务应用中, 有另外的方法用于处理不同线程中的指令错误。下表所列的特殊操作数与 XQ 指令配合使用就能使控制器跳过或重新处理无效指令。

特殊操作数

操作数	说明
_ED1	产生错误的线程号
_ED2	无效指令所在位置
_ED3	无效指令之后的指令所在位置

以下列格式与 XQ 指令一起使用操作数:

XQ_ED2 (或_ED3), _ED1, 1

其中: 在指令行末尾的 “, 1” 表示重新开始; 因此, 当执行上面的格式时, 不会去掉已有的程序堆栈。

下例示出使用操作数的错误修正子程序。

4) 指令错误 (使用多任务时)

指令	说明
#A	开始线程 0 (连续循环)
JP#A	
EN	结束
#B	开始线程 1
N=0	建立新变量
MT N	设置 MT 为 0 的值, 此为无效值
TY	无效指令
EN	结束
#CMDERR	开始指令错误子程序
IF_TC=6	如果错误为参数超出范围 (MT0)
N=2	设置 N 为有效数值
XQ_ED2, _ED1, 1	重执行 MT N 指令
ENDIF	
IF_TC=1	如果错误属无效指令(TY)
XQ_ED3, _ED1, 1	跳过无效指令
ENDIF	
EN	指令错误子程序结束

7) Ethernet 通信错误

在 DMC-B140 中运行这个简单程序并指出 (通过串口) 通信端口何时失败。通过监控串口, 若有必要, 用户能够重新建立通信。

指令	说明
----	----

#LOOP	简单程序循环
JP#LOOP	
EN	
#TCPERR	Ethernet 通信错误自动子程序
MG{P1}_IA4	将信息发送到串口，指明哪个端口没有接收正确的响应。
RE	

数学及函数表达式

数学运算符

为了进行数据运算变换，DMC-B140-M 提供如下数学运算功能

运算符	功能
+	加
-	减
*	乘
/	除
&	与
	或
()	括弧，用于改变运算顺序，可以嵌套

加、减、乘的运算数值范围为±2147483647.9999，除法运算的精确度为 1/65536。

注意：数学表达式的运算完全遵循从左向右的顺序，只有圆括弧可以改变这个顺序。

GALIL 并不遵守通常先乘除后加减的运算优先级顺序。

SPEED=7.5*V1/2	变量 SPEED 等于 V1 乘以 7.5 再除以 2
COUNT=COUNT+2	变量 COUNT 等于当前值加 2
RESULT=_TPA-(@COS[45]*40)	把 A 轴位置值减去 28.28 放入变量 RESULT。 40*COS45° 为 28.28
TEMP=@IN[1]&@IN[2]	只有当通用输入 1 和通用输入 2 均为截止状态时， 变量 TEMP 等于 1

位运算符

数学算子“&”和“|”是位运算符。运算符“&”是逻辑与，运算符“|”是逻辑或。这些运算符对任意有效的数字操作数（包括变量、数组元素、数字值、函数、操作数和算术表达式）进行位运算。位运算符也可以与字符串一起使用。这对于把输入字符串拆分为单个字符来说非常有用。采用输入指令进行字符串输入时，输入变量会拥有多达 6 个字符。这些字符合并成一个单值，表示为 32bit 整数和 16bit 小数。每个 ASCII 字符表达为 1 个字节(8bit)。字符串的第一个字符置于变量的最高字节，而最后一个字符放置于小数点后的最低有效字节。

能通过下例所示的位操作将字符逐个分开。

指令	说明
#TEST	开始主程序
IN “ENTER”, LEN{S6}	将 6 个字符的字符串放入变量“LEN”
FLEN=@FRAC[LEN]	将变量“FLEN”定义为变量“LEN”的小数

FLEN=\$10000*FLEN	部分 FLEN 左移 32 位 bits(将小数部分 FLEN 转换成整数))
LEN1=(FLEN&\$00FF)	屏蔽 FLEN 最高字节并将此值设置成变量“LEN1”
LEN2=(FLEN&\$FF00)/\$100	让变量“LEN2”=FLEN 的最高字节
LEN3=LEN&\$FF	让变量“LEN3”=LEN 的最低字节
LEN4=(LEN&\$FF00)/\$100	让变量“LEN4”=LEN 的第 2 字节
LEN5=(LEN&\$FF0000)/\$10000	让变量“LEN5”=LEN 的第 3 字节
LEN6=(LEN&\$FF000000)/\$1000000	让变量“LEN6”=LEN 的第 4 字节
MG LEN6{S4}	显示“LEN6”为 4 字符的字符串信息
MG LEN5{S4}	显示“LEN5”为 4 字符的字符串信息
MG LEN4{S4}	显示“LEN4”为 4 字符的字符串信息
MG LEN3{S4}	显示“LEN3”为 4 字符的字符串信息
MG LEN2{S4}	显示“LEN2”为 4 字符的字符串信息
MG LEN1{S4}	显示“LEN1”为 4 字符的字符串信息
EN	

此程序将接收 6 个字符的字符串输入，解析各个字符，然后显示各字符。也要注意，用于屏蔽的值以十六进制表示（由引导符号“\$”标注），对于更多的相关信息，参见“发送信息”一节。

为了进一步说明，如果用户在输入提示处输入字符串“TESTME”控制器就会响应如下：

T	来自指令 MG LEN6{S4}的响应
E	来自指令 MG LENS{S4}的响应
S	来自指令 MG LEN4{S4}的响应
T	来自指令 MG LEN3{S4}的响应
M	来自指令 MG LEN2{S4}的响应
E	来自指令 MG LEN1{S4}的响应

函数

函数	说明
@SIN[n]	正弦，n 以角度为单位
@COS[n]	余弦，n 以角度为单位
@TAN[n]	正切，n 以角度为单位
@ASIN[n]	反正弦，返回值在 $\pm 90^\circ$ 范围内，分辨率为 1/64000
@ACOS[n]	反余弦，返回值在 $0^\circ \sim 180^\circ$ 范围内，分辨率为 1/64000
@ATAN[n]	反正切，返回值在 $\pm 90^\circ$ 范围内，分辨率为 1/64000
@COM[n]	补码
@ABS[n]	绝对值
@FRAC[n]	取小数部分
@INT[n]	取整数部分
@RND[n]	四舍五入取整
@SQR[N]	平方根
@IN[n]	通用输入的状态（n 为 1~8）
@OUT[n]	通用输出状态（n 为 1~4）

函数可以与数学表达式结合使用，数学表达式的执行次序是从左到右，且能够用圆括号相括而优先执行。

指令	说明
V1=@ABS[V7]	变量 V1 等于变量 V7 的绝对值。
V2=5*@SIN[POS]	变量 V2 等于 5 倍变量 POS 的正弦
V3= @IN[1]	变量 V3 等于输入点 1 的数字值
V4=2*(5+@AN[5])	变量 V4 等于模拟输入点 5 的值加 5，然后乘以 2

变量

DMC-B140-M 允许用户使用最到 126 个独立变量，这些变量可以是数值或字符串。用户可以把某些参数（如位置或速度等）以变量形式写入程序；其后，再由操作人员分配变量值或者通过程序计算确定变量值。例如，定长裁断应用就需要切长为变量。

指令	说明
PR POSA	把变量 POSA 分配给 PR 指令
JG RPMB*70	把变量 RPMB 乘以 70 分配给 JG 指令

变量名

DMC-B140-M 允许用户创建 126 个变量，各变量可以用 8 个字符构成的变量名加以定义。变量名必须以英文字母开头，不过，在变量名的其余部分允许用数字，但不允许有空格。变量可以是大写或小写，或者任意组合。变量名对大小写敏感，SPEEDC 与 SpeedC 是两个不同的变量。变量名不宜与 DMC-B140 指令相同。例如，PR 就不适合用作变量名。

有效变量名：

POSA
POSI
SPEEDC

无效变量名

REALLONGNAME	不能多于 8 个字符
123	不能以数字开始
SPEED C	变量名中不能有空格

变量赋值

赋给变量的值可以是数字、其它变量、操作数、函数、控制参数和字符串；数值变量值的范围是 4 字节整数后跟 2 字节小数（±2, 147, 483, 647.9999）。

可以用等号把数值分配给变量，等号右边可以用任意有运算式。例如，V1=7+@ABS[V2] 或 V2=@IN[1]。

要分配字符串值，字符串必须在引号内。字符串变量最多可以含有 6 个字符。

指令	说明
POSA=_TPA	把从 TPA 指令返回的值分配给变量 POSA
SPEED=5.75	把 5.75 分配给变量 SPEED
INPUT=@IN[2]	把输入点 2 的逻辑值分配给变量 INPUT
V2=V1+V3*V4	将（V1+V3）*V4 的值分配给变量 V2，

VAR="CAT" 将字符串 CAT 分配给 VAR

以变量作为指令参数

可以将变量作为指令的数值参数

PR V1 将 V1 分配给 PR 指令
SP VSS*2000 将 VSS*2000 分配给 SP 指令

输出变量值

可以用格式：变量名=，使控制器输出变量的值。例如，

V1=

就会返回变量 V1 的值。

操作数

操作数允许把 DMC 控制器运动或状态参数带入表达式。大多数 DMC-B140 指令都有一个等效的操作数，通过在 DMC-B140 指令之前添加下划线即指定为操作数。《指令手册》中明确了哪些指令有相应的操作数。

查询指令，如 TP 读回实际位置值，而设置指令，如 KP 或 SP 就读回 DMC 寄存器中的值；指令之后需要指定轴。

指令	说明
POSA=_TPA	将来自 A 轴实际位置寄存器的值分配给变量 POSA.
JP#LOOP, _TEA>5	如果 A 轴位置误差寄存器中的值大于 5，就跳转到#LOOP
JP#ERROR, _TC=1	如果错误代码等于 1，就跳转到#ERROR

可以在表达式中使用操作数，并将其赋值给变量，但不能对其赋值，例如：_TPA=2 是无效的。

特殊操作数

DMC-B140-M 提供了一些附加操作数，用于存取那些不能用标准 DMC-B140-M 指令读取的内部状态。

关键字	说明
_BGn	如 n 轴运动完成, 返回 1, 否则为 0
_BN	控制卡的序号
_DA	可用数组变量数
_DL	可用标号数
_DM	可用的数组空间大小
_HMn	原点开关状态
_LFn	前极限开关状态
_LRn	后极限开关状态
_UL	可用变量数
TIME	上电后的伺服周期数（这是唯一不以下划线开头的操作数）

数组

为了便于采集、存储数据，DMC-B140 提供了 800 个元素的数组空间。数组是一维的，可以定义 6 个不同数组。各个数组元素值的范围与独立变量是一致的。可以用数组来捕获实时数据，如位置、转矩和模拟输入值。在轨迹方式下，在记录和录返应用中，可以很方便地用数组来保存位置轨迹的各点坐标值。

定义数组

用指令 DM 定义数组，用户必须指定数组名和元素数。数组名可以含有 8 个字符，以大写字母开头，再定义数组中的元素数，元素数在[]中。

DM POSA[7]	定义带有 7 个元素的数组名 POSA
DM SPEED[100]	定义带有 100 个元素的数组名 SPEED
DM POSA[0]	清空数组空间

数组元素赋值

像变量一样，可以给各数组元素赋值。分配的值可以是常数值或从任何运算表达式的值。数组元素地址从 0 开始，例如，POSA 数组中的第 1 个元素（用 DMPOSA[7]定义）就指定为 POSA[0]。

注意：在给数组赋值之前，必须用指令 DM 定义数组。

DM SPEED[10]	定义数组 Speed 大小
SPEED[0]=7650.2	给数组的第 1 个元素赋值 7650.2
SPEED[0]=	显示数组第一个元素的值
POSXA[9]=_TPA	给第 10 个元素赋值 A 轴实际位置值
CON[1]=@COS[POS]*2	给第 2 个数组元素赋值为 POS 的余弦值乘以 2
TIMER[0]=TIME	给第 1 个数组元素赋值为 TIME 值

用变量指定数组元素地址

数组元素地址也可以是一个变量。这样，就可以用一个计数器按序给数组元素赋值。

指令	说明
#A	程序标号
COUNT=0;DMPOS[10]	初始化计数器并定义数组
AT0	
#LOOP	开始循环
AT-10	等待 10msec
POS[COUNT]=_TPA	将位置值记录入数组元素
POS[COUNT]=	输出位置
COUNT=COUNT+1	计数器加 1
JP#LOOP, COUNT<10	循环，直到保存完 10 个元素
EN	结束

上例表示，以每 10ms 采集一个数值的速率将 10 个位置值存入 10 个数组元素，这 10 个值

存入数组名 POS 中，变量 count 用来使数组元素计数器增 1。上例也可用下述的自动数据采集功能来实现。

上载、下载数组到板上存储器

也可以用 QU 和 QD 指令上载和下载数组。

QUarray[], start, end, delim

QDarray[], start, end

其中：array 是数组名，例如：A[]；start 是第一个数组元素（缺省值=0）；end 是最后一个数组元素（缺省值：最后一个元素）；delim 定义数组数据用逗号“,”（delim=1）还是用回车（delim=0）来分开。用<control>Z, <control>Q, <control>D 或 \ 来终止文件。

将数据自动记录到数组中

DMC-B140-M 控制器有一个用于数据自动采集的特殊功能，可以用于对诸如位置、输入状态数据等进行自动采集，并记录在数组。这对于示教运动轨迹或观察系统性能而言非常有用。可以同时记录 4 组数据，并存入 4 个数组。可以指定采集速率（时间间隔），可以一次性或循环连续采集、记录数据。

指令汇总——自动数据记录

指令	说明
RA n[], m[], o[], p[]	为数据记录选择 4 个数组，之前必须用 DM 指令定义好数组
RD tp1, tp2, tp3, tp4	选择要记录的数据内容，其中：tp1, tp2, tp3, tp4 代表各种数据内容（见下表），数据类型的次序很重要，与 RA 指令中 n, m, o, p 数组的次序相对应。
RC n, m	RC 指令开始数据采集，设置数据采集时间间隔，其中：n 是 1-8 的整数，指定数据之间相隔 2 ⁿ 工作周期；m 是选择项，规定采集次数，若省略 m，就缺省为由数组的大小决定。当 m 是负数时，就以循环方式连续进行记录。_RD 是记录指针，指示下一个数组元素的地址。n=0 则停止记录。
RC?	返回值为 0 或 1，其中：0 表明不记录，1 表示记录在进行。
_RC	返回值为 0 或 1；0 表示不记录，1 表示记录在进行
_RD	返回值为下一个数组元素的地址

用于记录的数据

数据类型	说明
_SHn	规划位置
_RPn	规划位置
_DEn	发送脉冲位置
_TDn	发送脉冲位置
_TPn	编码器反馈位置
_TSn	开关点
_RLn	位置锁存的位置
_SCn	停止代码
_TI	通用输入状态
_OP	通用输出状态

n 为 A、B、C、或 D。

举例

指令	说明
#RECORD	标号
DM AP[300], BP[300]	定义 A、B 轴位置数组注
DM AP1[300], BP1[300]	定义 A、B 轴误差数组
RA AP[1], AP1[], BP[], BP1[]	为捕获选择数组
RD _TPA, _DEA, _TPB, _DEB	选择数据类型
PR l0000, 20000	指定运动距离
RC1	以 2msec 速率开始记录
BGAB	开始运动
#A: JP#A, RC=1	循环
MG"DONE"	输出信息
EN	结束程序
#PLAY	录返程序标号
N=0	初始化计数器
JP#DONE, N>300	如果完成, 退出
N=	输出计数器值
AP[N]=	输出 A 轴编码器位置
BP[N]=	输出 B 轴编码器位置
AP1[N]=	输出 A 轴输出脉冲位置
BP1[N]=	输出 B 轴输出脉冲位置
N=N+1	计数器加 1
#DONE	标号
EN	结束程序

取消数组空间分配

用 DA 指令后跟数组名就可以取消数组空间分配, DA*[0]将所有数组分配取消。

数据输入（数字和字符）

用指令 IN 来提示用户输入数字或字符串, 使用 IN 指令, 用户可以把信息放在双引号中, 指定信息提示内容。控制器执行 IN 指令时, 就会等待输入数据。把输入分配给所指定的变量或数组元素。

1) 数据输入

```
#A
IN "Enter Length", LenA
EN
```

在此例中, 控制器输出信息 “Enter Length”, 并等待输入一个长度值。操作人员输入的数字分配给变量 LenA。(注意: 在输入信息末尾的逗号和变量名之间不能有空格)。

2) 定长裁断

在此例中, 材料的长度要事先规定好, 当运动完成时, 使刀头工作, 切断材料。长度值是一个变量, 提示操作者以英寸输入。切割运动由连接到输入点 1 的启动按钮启动。负载与螺距为 2 英寸的丝杠相连接, 2000cts/rev 编码器安装在电机上, 分辨率为 4000cts/英寸。下面的程序对长度用变量 LEN, 用 IN 指令来提示操作人员输入长度值, 然后输入的长度值就分配给变量 LEN。

指令	说明
#BEGIN	标号
AC 800000	加速度
DC 800000	减速度
SP5000	速度
LEN=3.4	初始长度
#CUT	Cut 子程序
AI1	等待启动信号
IN"enter Length", LEN	提示操作人员以英寸为单位输入长度值
PR LEN*4000	以 cts 指定位置值
BG A	开始运动移动材料
AM A	等待运动完成
SB1	对切刀设置输出，使切刀动作
WT100;CB1	等待 100msec，然后关断切刀
JP#CUT	重复切割子程序
EN	结束

数据输出（数字和字符）

可以用几种方法从控制器输出数字和字符串数据。指令 **MG** 使控制器输出字符和数字数据。还有，能用对控制器发送查询指令进行指令，以返回变量和数组值以及其它信息。

输出信息

用指令 **MG** 可以将信息输出到通讯接口。这个指令可以输出数值、字符串，并指定输出的格式与使用的通讯接口。

用双引号指定文本字符串，而用变量或数组名指定变量或数组数据。例如：

MG"TheFinalValueis", RESULT

除变量、函数和指令之外，在信息指令中还可以使用操作数。例如：

MG"Analoginputis", @AN[1]

MG"The Position Of A is", _TPA

格式化输出信息

用{Sn}对字符串变量进行格式化，其中，n 是字符数，1-6。例如：

MG STR {S3}

这条语句返回字符串变量名为 **STR** 的 3 个字符。

用{Fn.m}跟在表达式后就可以对十进制数据进行格式化。{\$n.m}以十六进制而不是十进制对数据进行格式化。将实际数值格式化成小数点左面 n 个字符，小数点右面 m 个字符。**LZ** 指令可设置是否输出不必要的 0。

例如：

MG"The Final Value is", RESULT{F5.2}

如果变量 **RESULT** 的值等于 4.1，上述语句返回如下：

The Final Value is 00004.10

如果变量 **RESULT** 的值等于 999999.999，上述语句返回如下：

The Final Value is 99999. 99

信息指令通常发送跟着语句的回车和换行。在语句末尾发送{N}就可以阻止回车和换行。

当文本字符串需要处于数值前后时，这就很有用。

例如：

```
#A
JG 50000;BGA;ASA
MG"The Speed is",_TVA{F5. 1}{N}
MG"counts/sec"
EN
```

当执行上述程序时，屏幕上就会显示出：

```
The speed is 50000 counts / sec
```

用 MG 指令配置终端

能用 MG 指令对终端进行配置。可以用格式{^n}发送任意 ASCII 字符，其中：n 是 1~255 之间的任意整数。

例如：

```
MG{^07}{^255}
```

将 ASCII 码 7 和 255 的字符发送到总线。

信息函数格式化字符汇总

格式化字符	说明
“”	输出文本字符
{Fn.m}	格式化数值，小数点左面 n 位十进制数，右面 m 位
{P1}或{En}	发送到串口或网口
{\$n.m}	以 16 进制对数值进行格式化
{^n}	发送 ASCII 码为 n 的字符
{N}	阻止回车/换行
{Sn}	发送字符串的前 n 个字符

显示变量和数组

可以用格式，变量= 或数组[x]= 将变量或数组发送到屏幕上。例如，V1= 就发送 V1 的值。

举例——显示变量和数组元素

指令	说明
#DISPLAY	标号
DM POSA[7]	定义有 7 个元素的数组 POSA
PR l000	位置指令
BG A	开始运动
AM A	在运动完成后
V1=_TPA	变量 V1 赋值
POSA[1]=_TPA	第 1 个元素赋值
V1=	显示 V1
EN	

查询指令

控制器有一组直接查询控制器的指令。当输入这些指令时，就以十进制返回所要的数据，通过回车和换行而进入下一行。返回的数据格式能用位置格式化指令（PF）和前位充 0 指令（LZ）进行修改。关于查询指令的完整内容参见第 5 章。

用 PF 指令对查询指令所产生的回应进行格式化

PF 指令能修改由查询指令所返回值的格式：

BL?	LE?
DE?	PA?
DP?	PR?
EM?	VE?
FL?	TP
IP?	

用 PF 指令可以对十进制或十六进制数值格式化，使其该数的小数点右面和左面具有所规定的位数。指令格式如下：

PF m .n

其中：m 是小数点左面的位数（0-10），n 是小数点右面的位数（0-4），负的 m 则指定为十六进制格式。

返回十六进制值由\$开头，以 2 的补码表示。十六进制值应该按带符号 2 的补码进行输入，其中，负数有负号。缺省格式为 PF10.0。如果由 PF 所规定的十进制位数小于实际值，那么 9 就出现在所有的十进制位。

举例：

指令	说明
: DP21	定义位置
: TPA	报告位置
0000000021	缺省格式
: PF4	修改格式为 4 位
: TPA	报告位置
0021	新格式
: PF-4	修改成十六进制格式
: TPA	报告位置
\$0015	十六进制值
: PF2	修改格式为 2 位
: TPA	报告位置
99	如果位置值大于 99，就返回 99

从查询指令响应中去掉前面的 0

可以用指令 LZ 去掉由查询指令所返回值前面的 0。

指令	说明
LZ0	LZ 功能无效
TP	报告位置
-0000000009, 0000000005	返回值(带有前 0)
LZ1	LZ 功能使能
TP	报告位置
-9, 5	返回值（无前 0）

查询指令返回值的局部格式化

可以对查询指令返回值进行局部格式化。要想进行局部格式化，在查询指令的同一行使用指令{Fn.m}或{Sn.m}。符号 F 指出应该以十进制格式返回响应，而\$指定以十六进制返回。n 是小数点左面的位数，m 是小数点右面的位数。

指令	说明
TP{F2.2}	以十进制格式 2.2 报告位置
-05.00, 05.00, 00.00, 07.00	查询所得结果
TP{S4.2}	以十六进制格式 4. 2 报告位置
FFFB.00, \$0005.00, \$0000.00, \$0007.00	查询所得结果

对变量和数组元素格式化

用变量格式化(VF)指令对变量和数组元素进行格式化。

VF 指令格式如下：

VFm.n

其中：m 是小数点左侧位数（0-10），n 是小数点右侧位数（0-4）。m 的负号表示为十六进制格式。VF 的缺省格式为 VF10.4。

十六进制返回值前缀 S，以 2 的补码表示。

指令	说明
V1=10	V1 赋值
V1=	返回 V1
: 0000000010.0000	查得结果——缺省格式
VF2.2	更改格式
V1=	返回 V1
: 10.00	查得结果——新格式
VF-2.2	指定十六进制格式
VI=	返回 V1 值
\$0A.00	查得结果——十六进制值
VF1	修改格式
V1=	返回 V1 值
: 9	查得结果——溢出

变量的局部格式化

PF 和 VF 指令是广域格式化指令，它对所有相关返回值和变量的格式起作用。也可以对变量进行局部格式化。要想进行局部格式化，就使用指令{Fn.m}或{Sn.m}后跟变量名和“=”。F 规定十进制，\$规定十六进制。n 是小数点左侧位数，m 是小数点右侧的位数。

指令	说明
V1=10	V1 赋值
V1=	返回 V1 值
: 0000000010.0000	缺省格式
V1={F4.2}	规定局部格式
: 0010.00	新格式值
V1={S4.2}	规定十六进制局部格式
: \$00A0.00	十六进制值
V1= “ALPHA”	V1 赋值为 “ALPHA”
V1={S4}	指定字符串为前 4 个字符
: ALPH	

输入中断功能

DMC-B140-M 提供有输入中断功能，此功能会使程序自动执行跟在#ININT 标号之后的指令。用 II m, n, o 指令使此功能有效。m 代表起始输入，n 代表范围内的最后输入，参数 o 是中断屏蔽。如果不用 m 和 n，o 就表示屏蔽号。1 表示对中断进行使能的输入，其中， 2^0 是 bit1， 2^1 是 bit2，以此类推。例如，II, , 5 使输入点 1 和 3 ($2^0+2^2=5$) 有效。

任何一个指定输入的导通输入都会使#ININT 子程序自动执行。用中断返回指令（RI）使程序运行从这个子程序返回到中断发生的地方。如果在执行#ININT 子程序之后，要想返回到程序中的其它地方，就用零堆栈（ZS）指令后面跟着无条件跳转语句。

重要提示：要从#ININT 子程序返回，使用 RI 指令（而不是 EN）

举例——输入中断

指令	说明
#A	标号#A
II1	由通用输入 1 产生中断
JG30000, -20000	设定 A、B 轴速度
BG AB	开始 A、B 轴运动
#B	标号#B
TP AB	报告 A、B 轴位置
WT 1000	等待 1000ms
JP#B	跳转到#B
EN	程序结束
#ININT	中断处理子程序
MG"Interrupt has occurred"	显示信息
ST AB	停止 A、B 轴运动
#LOOP;JP#LOOP, @IN[1]=0	循环直至中断清除
JG 15000, 10000	指定新速度
WT 300	等待 300ms
BG AB	开始 A、B 轴运动
RI	中断返回

应用举例

线缆裁切器

操作人员按下启动开关，使电机带动线缆前进 250 毫米，当运动停止时，控制器产生输出信号，启动切刀。切割动作在 100ms 周期内完成。

假设电机通过一个直径为 50 毫米的辊子驱动线缆，该电机 4000 个脉冲转 1 圈。因此，250 毫米行程就等于：

$$4000/50 \pi * 250 = 6366 \text{ 脉冲}$$

如运动速度为 125mm/sec，即等于 3183 脉冲/sec。

启动开关连接到通用输入 1，而切刀控制连接到通用输出 1，电机速度曲线和相关输入、输出信号示于下图。

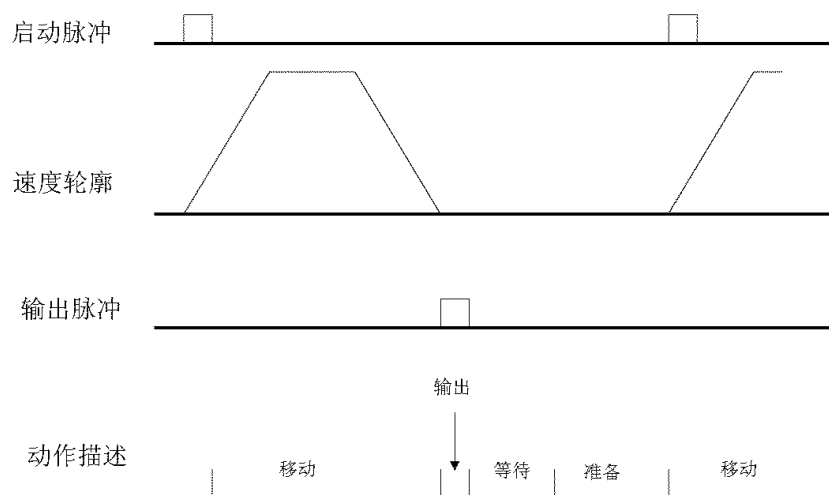


图 7.1 -电机速度和相关输入/输出信号

程序按我们在程序#A 中所定义的状态起动，这里，控制器等待通用输入 1 的输入脉冲，一旦脉冲收到，控制器就开始正向运动。

正向运动一完成，控制器就输出 20ms 的脉冲，然后在返回#A 进行下一个新循环之前等待另外 80ms。

指令	说明
#A	标号
AI1	等待通用输入 1
PR 6366	距离
SP 3183	速度
BGA	开始运动
AMA	运动完成后
SB1	通用输出 1 导通
WT 20	等待 20ms
CB1	通用输出 1 截止
WT 80	等待 80ms
JP#A	重复循环

X-Y-Z 工作台控制器（3 轴应用）

一台 X-Y-Z 三轴系统必须切割出下图所示的形状，X-Y 工作台做平面运动，而 Z 轴使切割刀具做上、下运动。

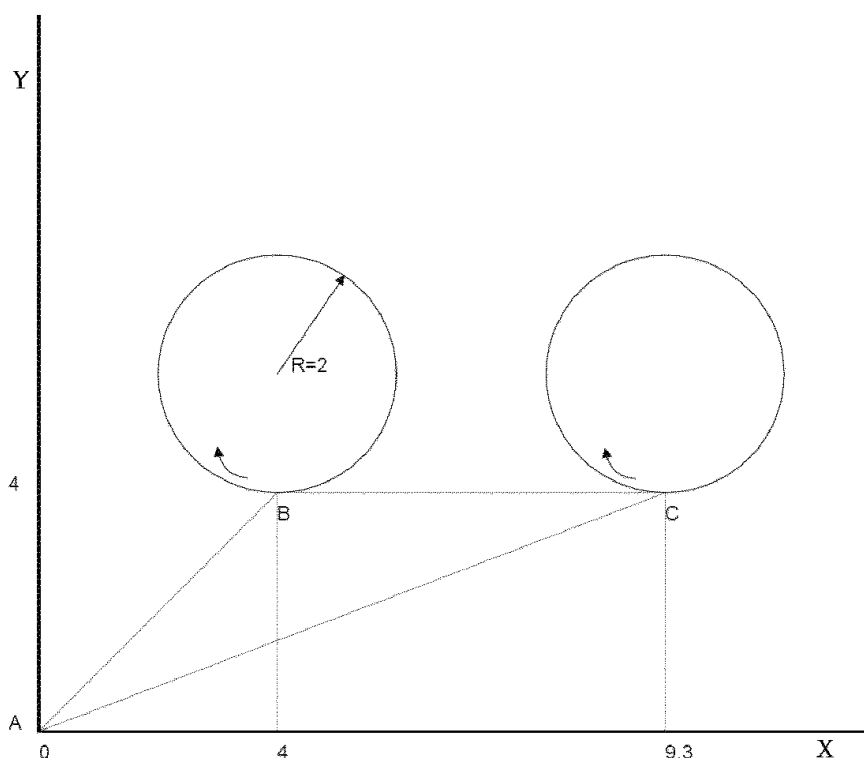


图 7.2 -XY 平面内的切割轨迹

图 7.2 中的实线表示刀具行走的切割路径，要求进给速率为 1 inch / sec；虚线表示刀具的非切割路径，运动速率为 5 inch/sec，加速度为 0.1G。

运动从 A 点开始，Z 轴抬起，X-Y 轴移动到 B 点的同时，Z 轴向下运动，开始沿圆弧进行切割。圆弧加工运动一完成，Z 轴上升抬起刀具，X、Y 轴继续运动到 C 点，以此类推。

假定，3 个轴的丝杆螺距都为 0.1 inch/rev，且电机 4000 脉冲 / rev，由此得出：

1 inch=40,000 脉冲

速度：1 in / sec=40,000 脉冲/sec；5 in/sec=200,000 脉冲/sec

加速度：0.1G=38.6 in/sec²=1,544,000 脉冲/sec²

注意：圆弧半径为 2 英寸或 80000 脉冲，加工起始点角度为 270°，以 CW（逆时针方向）旋转 360°。这样，就用如下指令规定轨迹：

CR80000, 270, -360

进一步假定，Z 轴移动 2 英寸，移动速度为 2 英寸/sec。用以下指令完成所要求的运动：

指令	说明
#A	标号
VM AB	X、Y 轴矢量插补
VP 160000, 160000	矢量位置
VE	结束矢量运动
VS 200000	矢量速度
VA 1544000	矢量加速度
BGS	开始运动
AMS	运动完成时
PR, , -80000	Z 轴下降
SP, , 80000	Z 轴速度
BG C	开始 Z 轴运动
AM C	等待 Z 轴运动完成
CR 80000, 270, -360	圆弧

VE	结束矢量运动
VS 40000	进给率
BGS	开始圆弧运动
AMS	等待圆弧运动完成
PR, , 80000	抬起刀具（Z 轴）
BG C	开始 Z 轴运动（抬刀）
AM C	等待 Z 轴运动完成
PR -21600	移动 X 轴
SP 20000	X 轴速度
BGA	开始 X 轴运动
AMA	等待 X 轴运动完成
PR, , -80000	落下 Z 轴（下刀）
BG C	
AM C	
CR 80000, 270, -360	第 2 个圆弧切割
VE	
VS 40000	
BG S	
AM S	
PR, , 80000	抬刀
BG C	
AM C	
VP -37600, -16000	X、Y 轴返回到起点
VE	
VS 200000	
BG S	
AM S	
EN	

第八章 防护

说明

DMC-B140-M 提供了几种硬件、软件保护功能，以检查外部错误，并禁止电机运动。这些功能有助于保护各种系统元件免受损坏。

警告：机器在运动中可能有危险！用户有责任设计有效的报警处理及安全保护。由于 DMC-B140-M 是机器中的一个集成部分，因此，工程师应该设计具有保护措施完整系统，以免造成 DMC-B140-M 元件损坏。

硬件防护

DMC-B140-M 有硬件输入和输出保护，用于报警和机械限位保护。

输出保护

放大器使能：控制器可以通过这个输出关闭电机。除直接发送 MO 指令外，还有其它一些条件都会使控制器关闭电机：

- (1) 给出急停命令时。
- (2) 当看门狗定时器触发或复位时，此信号也变低从而关断伺服输出。

报警输出：在 DMC-B140-M 控制器中，提供报警指示灯（红色）与控制器就绪输出。正常情况下，报警指示灯不发光，就绪输出导通。以下几种情况可能导致报警指示灯发光，就绪输出截止：

- (1) 控制器侧的复位按键被按下。
- (2) 控制器有故障且处理器正在自身复位。
- (3) 驱动报警信号的输出 IC 芯片有故障。
- (4) 控制器外用户提供的 DC5V 电源故障

输入保护

通用急停 (ABORT)：此输入信号有效，就立即停止各轴运动，而且是不经过减速处理的停止。对于报警关断功能使能的任何轴来说，都会使其使能信号关断。这样会使电机依靠自身惯性自由滑行而停止。如果没有使报警关断功能使能，电机将立即停止，锁定在当前位置（但有可能产生抖动或震荡）。

单轴急停：也可以对控制器进行配置，对各轴提供各自的急停。其作用与 ABORT 输入相同，但只对指定轴有效。要想把控制器配置成选择性急停，使用命令 CN, , 1 即可。这样就将输入点 5, 6, 7, 8 配置成分别对应轴 A, B, C, D 各自的急停。

正向位置极限信号：此信号有效禁止正向运动。当此信号有效时，电机正在向正方向运动，那么，就会使运动减速而停止。此外，如果控制器正在执行用户程序，就会自动跳转到位置极限子程序#LIMSWI。可以用 CN 命令改变位置极限信号的极性。

反向位置极限信号开关：此信号有效禁止反向运动。当此信号有效时，电机正在向反方向运动。那么，就会使运动减速而停止。此外，如果控制器正在执行用户程序，就会自动跳转到位置极限子程序#LIMSWI（在用户将此子程序写入应用程序的情况下）。可以用 CN 命令改变信号极性。

软件防护

位置限制（软件限位）功能

DMC-B140-M 具有软件限位功能，可以用 BL 和 FL 软件命令对正、反向软件限位值进行设置。一旦指定限位值，DMC-B140-M 就不会接受超过此限制值的位置命令。当然，也防止了超过限制值的运动。

举例

指令	说明
DP0, 0, 0	定义位置
BL-2000, -4000, -8000	设定反向位置限制值
FL 2000, 4000, 8000	设定正向位置限制值
JG 2000, 2000, 2000	Jog 运动
BG ABC	开始
运行此程序，运动停止在正向限制值处	

位置极限信号处理子程序

DMC-B140-M 具有硬件正、反向限位功能。对于位置极限信号子程序，也有一个特殊标号#LIMSWI。标号指定开始位置极限信号子程序。如果任何一个位置极限信号有效，而且该轴电机正在以同方向运动，那么此标号就便跟随其后的语句自动被执行。RE 命令使此子程序结束。在条件跳转期间，也可以测试正、反向位置极限信号状态。_LR 指定反向位置极限信号，而_LF 指定位置极限信号。跟随 LR 或 LF 的 A, B, C, D 指定轴。用 CN 命令配置位置极限信号的极性。

举例：

指令	说明
#A; IP#A; EN	虚拟程序
#LIMSWI	位置极限信号处理子程序
V1=_LFA	检查是否正向位置极限信号
V2=_LRA	检查是否反向位置极限信号
JP#LF, V1=0	如果是正向位置极限信号有效，跳转到#LF
JP#LR, V2=0	如果是反向位置极限信号有效，跳转到#LR
JP#END	跳转到#END
#LF	标号
MG"FORWARD LIMIT"	发送信息
STA; AMA	停止运动
PR-1000; BGA; AMA	反向运动
JP#END	跳转到#END
#LR	标号

MG"REVERSE LIMIT"	发送信息
STA;AMA	停止运动
PR 1000;BGA;AMA	正向运动
#END	结束
RE	返回到主程序

注意：只有在应用程序执行期间，#LIMSWI 才会起作用。

附录

DMC-B140 控制器数据记录集

DMC-B140 网络 I/O 扩展

DMC-B140 控制卡规格